



Facultad de Ciencias

**Diseño e Implementación de un Generador de
Buscadores de Pares de Legendre mediante
Metaprogramación y Computación de Alto
Rendimiento**

Design and Implementation of a Legendre Pairs
Researcher Generator using Metaprogramming and High
Performance Computing

Trabajo de fin de grado
para acceder al

GRADO EN INGENIERÍA INFORMÁTICA

Autor: Carlos Alarcón Gutiérrez
Director: Domingo Gómez Pérez

septiembre de 2025

Agradecimientos

A mi familia

Resumen

palabras clave: Pares de Legendre, Metaprogramación, OpenMP, Slurm, HPC, Optimización

Los pares de Legendre son pares de secuencias de longitud ℓ , formadas por ceros y unos que cumplen una relación entre las funciones de autocorrelación. Esta relación en las funciones de autocorrelación es la base del uso de estas secuencias en aplicaciones críticas como sistemas de posicionamiento y sistemas criptográficos. A pesar del interés suscitado tanto en la industria como dentro de la investigación, actualmente no se conoce ninguna fórmula cerrada para generarlas para un valor general ℓ .

Este Trabajo de Fin de Grado aborda el problema de la búsqueda de pares de Legendre bajo una nueva conjetura. Aunque esta conjetura simplifica la búsqueda de estos pares de secuencias, el problema sigue siendo un desafío de optimización combinatoria caracterizado por un crecimiento exponencial del espacio de búsqueda, lo que lo hace inabordable mediante una exploración exhaustiva. El objetivo principal es diseñar e implementar un sistema automatizado capaz de generar y ejecutar programas especialmente diseñados para la detección de estos pares, haciendo uso de técnicas avanzadas de metaprogramación y computación de alto rendimiento (HPC). Para ello, se propone una solución integral que combina diferentes técnicas: reducción algebraica mediante cosets ciclotómicos y compresión, junto con algoritmos de búsqueda en profundidad con poda dinámica, estructuras eficientes basadas en bitsets y estrategias probabilísticas como filtros de Bloom en un esquema meet-in-the-middle. Estas técnicas permiten reducir drásticamente la complejidad del problema, tanto en tiempo como en memoria.

*Design and Implementation of a Legendre Pairs Researcher Generator using
Metaprogramming and High Performance Computing*

Abstract

keywords: Legendre Pairs, Metaprogramming, OpenMP, Slurm, HPC, Optimization

This project addresses the problem of searching for Legendre pairs under a novel conjecture. Although this conjecture simplifies the search for such sequence pairs, the problem remains a challenging combinatorial optimization task, characterized by the exponential growth of the search space, which renders exhaustive exploration infeasible. The main objective of this work is to design and implement an automated system capable of generating and executing specialized programs for detecting these pairs, leveraging advanced metaprogramming techniques and high-performance computing (HPC). To this end, a comprehensive solution is proposed that combines several approaches: algebraic reduction through cyclotomic cosets and compression, together with depth-first search algorithms incorporating dynamic pruning, efficient bitset-based data structures, and probabilistic strategies such as Bloom filters within a meet-in-the-middle framework. These techniques enable a substantial reduction in both time and memory complexity. All these methods are implemented in a C++ code generator that, given a set of input parameters, automatically produces optimized executables. The system exploits parallelism at multiple levels, using OpenMP for shared-memory parallelism and Slurm for task distribution in supercomputing environments. This approach maximizes the utilization of modern architectures, including vectorization and superscalar execution. Experiments carried out on the Altamira supercomputer demonstrate the effectiveness of the system, successfully solving instances of significant size ($\ell = 75$) and yielding new valid Legendre pairs, verified both analytically and computationally.

Índice

1. Introducción	1
1.1. Antecedentes históricos y relevancia actual	1
1.2. Aplicaciones y logros de los pares de Legendre	3
1.2.1. Sistemas de radar y sonar	3
1.2.2. Diseño de experimentos y matrices ortogonales	4
1.2.3. Criptografía y secuencias pseudoaleatorias	4
1.2.4. Sistemas de posicionamiento satelital (GPS) y localización UWB	4
1.2.5. Análisis de Funciones Booleanas mediante la Transformada de Walsh-Hadamard	5
1.3. El reto computacional y la combinatoria explosiva	5
1.3.1. Fundamentos Matemáticos del Método de Compresión	6
1.4. Propiedades matemáticas fundamentales	7
1.4.1. Función de Autocorrelación Periódica (PAF)	7
1.4.2. Densidad Espectral de Potencia (PSD)	8
1.5. Objetivos y metodología de desarrollo	8
1.5.1. Metodología en Cascada (Waterfall)	8
1.5.2. Herramientas y Decisiones Arquitectónicas	8
1.5.3. Estructura de la Memoria	9
2. Desarrollo e Innovaciones Algorítmicas	11
2.1. Reducción Algebraica: Cosets Ciclotómicos	11
2.1.1. Herramientas y Decisiones Arquitectónicas	13
3. Desarrollo e Innovaciones Algorítmicas	15
3.1. Reducción Algebraica: Cosets Ciclotómicos	15
3.2. Evolución del Software: De la I/O a la Búsqueda en Profundidad	15
3.2.1. Fase 1: Fuerza Bruta sobre Cosets y Archivos Intermedios	16
3.2.2. Fase 2: Búsqueda en Profundidad (DFS) y Podas (Pruning)	16
3.2.3. Fase 3: El Estado del Arte (Bitsets y Ordenación Heurística)	18
3.3. Reducción de Complejidad Extrema: Filtros de Bloom y Estrategia Meet-in-the-Middle	19
3.4. Estrategia de Metaprogramación: El Generador en C++	21
3.4.1. Cálculo Simbólico y Arquitectura Superescalar (Loop Unrolling)	21
3.4.2. Sistema de División por Lotes Integrado (Batching)	23
3.5. Automatización del Flujo y Benchmarking: El Makefile	24
4. Resultados y Análisis Experimental	27
4.1. Entorno Experimental: Supercomputador Altamira	27
4.2. Análisis Teórico de Escalabilidad (Ley de Amdahl)	28
4.3. Análisis de Rendimiento Empírico	29
4.3.1. Impacto de la Solución de Lotes (Slurm Arrays)	29
4.3.2. Impacto del Desenrollado y Vectorización	29
4.4. Hallazgos Matemáticos: Secuencias Encontradas	30

5. Conclusiones y Trabajo Futuro	33
5.1. Conclusiones Clave	33
5.1.1. La Metaprogramación como Técnica Habilitante de Rendimiento Extremo . . .	33
5.1.2. Transformación de Complejidad mediante Estructuras Probabilísticas	33
5.1.3. Sinergia Arquitectónica: Paralelismo Híbrido en Entornos HPC	34
5.2. Líneas Futuras de Investigación	34
5.2.1. Migración de Arquitectura y Portabilidad a GPGPU (CUDA)	34
5.2.2. Exploración de la Frontera Analítica Inexplorada ($\ell = 117$)	35
5.2.3. Democratización del HPC: Interfaces de Software como Servicio (SaaS)	35
Referencias	37

1 Introducción

En la actualidad, los sistemas de comunicación inalámbrica constituyen la columna vertebral de la infraestructura tecnológica global. Desde la telefonía móvil de última generación (5G/6G) y las redes Wi-Fi de alta velocidad hasta la transmisión satelital y los sistemas de radar de apertura sintética (SAR), la capacidad de transmitir información de manera fiable sin soporte físico ha revolucionado la interacción humana y el acceso masivo a datos. No obstante, estos sistemas operan en un entorno físico intrínsecamente hostil: el canal de transmisión es ruidoso, sujeto a interferencias multi-trayecto, atenuaciones por distancia y distorsiones de fase que comprometen la integridad de la señal.

Adicionalmente, el espectro electromagnético es un recurso finito y altamente disputado, hay bandas de frecuencia congestionadas y reguladas, como se puede ver en [este gráfico](#), lo que obliga a los ingenieros a diseñar señales que maximicen la eficiencia espectral. Para mitigar estos efectos, las señales analógicas se digitalizan, transformándose en secuencias discretas de información. Cada secuencia se transmite en «slots» de tiempo o en bandas de frecuencia específicas, y su estructura interna determina la capacidad del sistema para resistir el ruido, evitar interferencias y mantener la confidencialidad. Sabiendo el periodo de la secuencia, ℓ^1 , la señal se modela como un vector con ℓ de bits, donde cada posición representa un estado de la señal (por ejemplo, presencia o ausencia de una portadora). La distancia entre dos señales se puede medir mediante el producto escalar, lo que a su vez se relaciona con la función de autocorrelación de la secuencia. La función de autocorrelación periódica son los diferentes valores del producto escalar al desplazar la secuencia sobre sí misma.

Esta abstracción permite aplicar el vasto arsenal del álgebra abstracta y la combinatoria para el diseño de códigos. En este contexto, la teoría de secuencias binarias con propiedades óptimas de autocorrelación y espectro plano es un área de investigación crítica y activa. Entre estas, los *Pares de Legendre* destacan por su estrecha vinculación con las matrices de Hadamard y su aplicabilidad directa en la inmunidad al ruido en sistemas de radar y comunicaciones seguras.

1.1. Antecedentes históricos y relevancia actual

[Comentario: Añade referencias históricas sobre las matrices de Hadamard]

El estudio de secuencias con autocorrelación ideal tiene sus raíces en los problemas de diseño combinatorio del siglo XX. Las primeras referencias sobre las matrices de Hadamard aparecen en 1867, cuando el matemático J. J. Sylvester introdujo la construcción recursiva de estas matrices y que maximizaban el determinante de matrices con entradas ± 1 . Hadamard posteriormente generalizó esta idea en 1893, estableciendo el límite superior para el determinante de matrices binarias y demostrando que si teníamos una matriz H de orden n con entradas en $\{-1, 1\}$, entonces $|\det H|^2$ era n^n si y solo si H era una matriz de Hadamard, es decir, si se cumplía la siguiente relación de ortogonalidad:

$$HH^T = nI_n, \quad (1)$$

donde I_n es la matriz identidad de orden n . Hadamard conjeturó que estas matrices solo existen para órdenes n que son múltiplos de 4, lo que se conoce como la Conjetura de Hadamard, un problema abierto hasta el día de hoy y el primer valor que no se sabe si existe es $n = 668$.

¹La notación es estándar y representa la longitud

Una forma de construir matrices de Hadamard de orden $2\ell+2$ es a través de la construcción de dos circulantes (*two-circulant core construction*), que se basa en encontrar un par de secuencias binarias A y B de longitud ℓ .

$$\begin{aligned} A &= (a_0, a_1, \dots, a_{\ell-1}), \quad a_i \in \{-1, 1\} \\ B &= (b_0, b_1, \dots, b_{\ell-1}), \quad b_i \in \{-1, 1\} \end{aligned}$$

Ahora definimos las matrices circulares $\mathbf{A}, \mathbf{B}$ de la siguiente forma:

$$\mathbf{A} = \begin{bmatrix} a_0 & a_{\ell-1} & \dots & a_1 \\ a_1 & a_0 & \dots & a_2 \\ \vdots & \vdots & \ddots & \vdots \\ a_{\ell-1} & a_{\ell-2} & \dots & a_0 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} b_0 & b_{\ell-1} & \dots & b_1 \\ b_1 & b_0 & \dots & b_2 \\ \vdots & \vdots & \ddots & \vdots \\ b_{\ell-1} & b_{\ell-2} & \dots & b_0 \end{bmatrix}$$

Ahora utilizamos la comilla simple para denotar la matriz transpuesta, es decir, $\mathbf{A}'$ es la transpuesta de $\mathbf{A}$ y $\mathbf{B}'$ es la transpuesta de $\mathbf{B}$. La matriz de Hadamard se construye a partir de estas secuencias como sigue:

$$H = \begin{bmatrix} -1 & -1 & \mathbf{1} & \mathbf{1} \\ -1 & \mathbf{1} & \mathbf{1} & -\mathbf{1} \\ \mathbf{1}' & \mathbf{1}' & \mathbf{B} & \mathbf{A} \\ \mathbf{1}' & -\mathbf{1}' & \mathbf{A}' & -\mathbf{B}' \end{bmatrix}$$

Con esta construcción, la matriz H es de orden $2\ell+2$ y es una matriz de Hadamard si y solo si las secuencias A y B cumplen la siguiente condición:

$$\text{PAF}_A(k) + \text{PAF}_B(k) = \sum_{i=0}^{\ell-1} (a_i a_{i+k}) + \sum_{i=0}^{\ell-1} (b_i b_{i+k}) = -2, \quad \forall k \neq 0 \quad (\text{mód } \ell),$$

donde los índices se toman módulo ℓ . A estas secuencias A y B que cumplen esta condición se les denomina *Pares de Legendre*. **[Comentario: Podrías añadir aquí una breve explicación de qué es la función de autocorrelación periódica (PAF), cuanto es el valor máximo y comprueba los cálculos de la tabla del ejemplo.]**

Ejemplo de Pares de Legendre

Tomemos las secuencias $(1, 1, 1, -1, 1, -1, 1, -1, -1)$, $(1, 1, 1, 1, -1, -1, 1, -1, -1)$, calculemos la suma de sus autocorrelaciones periódicas, obtenemos:

k	$\text{PAF}_A(k)$	$\text{PAF}_B(k)$
0	9	9
1	-3	1
2	1	-3
3	-3	1
4	1	-3
5	1	-3
6	-3	1
7	1	-3
8	-3	1

La matriz de Hadamard de orden $2\ell+2 = 20$ se construye y no se incluye por razones de espacio.

Aunque el nombre “Pares de Legendre” evoca al matemático Adrien-Marie Legendre debido al uso de los símbolos de residuos cuadráticos, su definición moderna en el contexto de procesamiento de señal es posterior.

Históricamente, la investigación se centró en encontrar una única secuencia con autocorrelación ideal, lo que llevó a la búsqueda de secuencias pseudoaleatorias de Barker, que solo existen para longitudes muy cortas ($\ell \leq 13$). La inexistencia para longitudes impares mayores que 13 motivó la búsqueda de *pares* de secuencias complementarias. Investigadores como [Seberry y Yamada \[1992\]](#) formalizaron la conexión entre estos pares y las Matrices de Hadamard.

Recientemente, el trabajo de [Kotsireas y Koutschan \[2021\]](#) revitalizó el campo aplicando optimización computacional para longitudes ℓ divisibles por 3, ya que el primer caso desconocido de matrices de Hadamard ($n=668$) corresponde a encontrar un par de longitud $\ell = 9 \cdot 31 = 333$. Este trabajo mejora los resultados computacionales y ha hallado una modernización de las herramientas de búsqueda mediante paralelismo masivo y generación automática de código.

1.2. Aplicaciones y logros de los pares de Legendre

La relevancia de estas secuencias trasciende la teoría matemática pura; poseen aplicaciones prácticas directas en ingeniería moderna.

1.2.1. Sistemas de radar y sonar

En el diseño de ondas para radar, el objetivo fundamental es lograr una Función de Ambigüedad que se aproxime a un impulso ideal. La capacidad de discernir el eco de un objetivo frente al ruido depende de la Función de Autocorrelación Periódica (PAF) de la señal emitida.

Si transmitimos una secuencia S , buscamos que el lóbulo principal en el retardo $\tau = 0$ contenga toda la energía, y que los lóbulos laterales (*sidelobes*) sean nulos para evitar objetivos fantasma. Los pares de Legendre son valiosos porque, al transmitirse en esquemas ortogonales complementarios, la suma de sus interferencias se cancela casi perfectamente:

$$\text{PAF}_A(\tau) + \text{PAF}_B(\tau) = \begin{cases} 2\ell, & \tau \equiv 0 \pmod{\ell} \\ -2, & \tau \not\equiv 0 \pmod{\ell} \end{cases}$$

Esta propiedad de autocorrelación maximiza la Relación Señal a Ruido (SNR), mejorando la precisión del radar y mitigando las interferencias de trayectos múltiples (*multipath fading*) [Budišin \[1991\]](#).

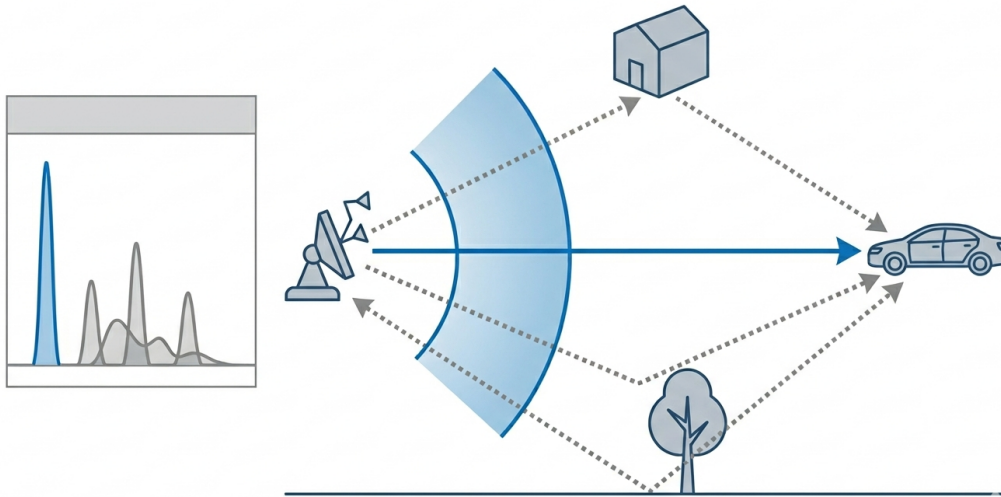


Ilustración 1: Visualización del fenómeno de desvanecimiento por trayectos múltiples (*multipath fading*). La capacidad del receptor para aislar el pulso directo frente a los ecos parásitos depende críticamente de la autocorrelación de secuencias ortogonales como los pares de Legendre.

1.2.2. Diseño de experimentos y matrices ortogonales

En estadística experimental, las matrices ortogonales se utilizan para estudiar el efecto de múltiples variables de forma simultánea, garantizando que la varianza sea mínima y los efectos independientes. La estructura óptima para esto es la Matriz de Hadamard, definida en la ecuación (1). **[Comentario: Hay aquí repetición de cosas, pero hay que explicar mejor porque las matrices ortogonales son importantes en el diseño de experimentos.]**

1.2.3. Criptografía y secuencias pseudoaleatorias

En sistemas de telecomunicaciones seguros, como el espectro ensanchado (Spread Spectrum) y los cifradores de flujo (*stream ciphers*), es crucial que la señal portadora sea estadísticamente indistinguible del ruido blanco gaussiano para evitar la interceptación y el criptoanálisis.

Por el Teorema de Parseval, la energía total de una señal en el dominio del tiempo es igual a su energía en el dominio de la frecuencia. **[Comentario: Podrías añadir aquí una breve explicación de qué es la Densidad Espectral de Potencia (PSD) y cómo se relaciona con la PAF,]**

Un par de Legendre asegura una Densidad Espectral de Potencia (PSD) sumada estrictamente constante para todo componente frecuencial k :

$$\text{PSD}_A(k) + \text{PSD}_B(k) = 2\ell + 2$$

Esta uniformidad espectral (espectro plano) dota a las secuencias de Legendre de un alto nivel de entropía, baja predictibilidad lineal y un comportamiento cuasi-aleatorio perfecto, haciéndolas ideales como firmas digitales u ondas portadoras inmunes a ataques por análisis de frecuencia Damgård [1988].

1.2.4. Sistemas de posicionamiento satelital (GPS) y localización UWB

El auge de los sistemas de localización, tanto en exteriores (sistemas satelitales GNSS/GPS) como en interiores mediante señales de radio de impulso ultraancho (IR-UWB), exige una precisión sub-métrica en el cálculo del Tiempo de Llegada (ToA) y la Diferencia de Tiempo de Llegada (TDoA). **[Comentario: Antes de esto tendrías que explicar brevemente qué es el ToA y el TDoA, y por qué es tan importante la precisión en estos sistemas. Como se calcula la distancia a partir del ToA y TDoA, y cómo el ruido afecta a esta]**

En estos entornos industriales o urbanos, las señales electromagnéticas se enfrentan a un canal hostil caracterizado por el desvanecimiento por trayectos múltiples (*multipath fading*), donde los ecos de la señal rebotan en obstáculos y confunden al receptor Pérez [2021].

Para mitigar este efecto y aislar la señal directa real, los sistemas modernos rechazan las modulaciones clásicas en favor del Acceso Múltiple por División de Código (CDMA)

o modulaciones por posición de pulso combinadas con saltos temporales (TH-PPM). Es aquí donde los Pares de Legendre demuestran una utilidad excepcional.

Al emplear estas secuencias combinatorias para modular los pulsos, se aprovecha su propiedad de autocorrelación casi ideal. El receptor, que conoce la secuencia de antemano, realiza una correlación cruzada con la señal entrante. Gracias a la estructura de Legendre, el pico de correlación en el retardo exacto $\tau = 0$ es agudo, mientras que los ecos desplazados caen en lóbulos laterales nulos o mínimos. Esto permite identificar el instante exacto del impacto del pulso principal con precisión de picosegundos.

Adicionalmente, el uso de familias de secuencias pseudoaleatorias derivadas de Legendre asegura una baja correlación cruzada entre distintos transmisores. Esto soluciona dos problemas críticos del mercado actual del posicionamiento:

- **Escalabilidad:** Permite que decenas de etiquetas móviles (MUs) y puntos de acceso (APs) compartan el mismo ancho de banda sin colisiones destructivas.
- **Seguridad de capa física:** Previene ataques activos de interferencia (*jamming*) y suplantación (*spoofing* o ataques *Early-Detection/Late-Commit*). Al ser el espectro de la señal estadísticamente

indistinguible del ruido blanco para un interceptor que desconoce la secuencia, el atacante no puede inyectar energía maliciosa de manera coherente para alterar la estimación de distancia.

[Comentario: He puesto esta sección aquí porque es una aplicación directa de las matrices de Hadamard. Pero ahora creo que hay notación que se repite y no se ponen ejemplos concretos. Así queda muy abstracto y difícil de entender. Podrías poner un ejemplo concreto de cómo se construyen las matrices de Sylvester.]

1.2.5. Análisis de Funciones Booleanas mediante la Transformada de Walsh-Hadamard

En el ámbito de la criptografía, los Pares de Legendre y las secuencias de espectro plano están íntimamente ligados a la robustez de las funciones booleanas empleadas en S-boxes (para cifradores de bloque) y en combinadores de registros de desplazamiento (para cifradores de flujo). Una función booleana $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ debe exhibir la máxima no linealidad posible para frustrar el criptoanálisis lineal y diferencial.

La herramienta matemática estándar para desnudar y cuantificar cualquier linealidad oculta en estas funciones es la Transformada de Walsh-Hadamard (WHT). Conceptualmente, la WHT es el caso especial de la Transformada Discreta de Fourier sobre el grupo abeliano finito $\mathbb{F}_2^n$.

Para calcular el espectro de Walsh de una función, se multiplica su vector de verdad polarizado por una Matriz de Hadamard de orden 2^n . Sin embargo, es un requisito matemático estricto que dicha matriz se genere exclusivamente mediante la **construcción de Sylvester**. Esta matriz se define de forma recursiva a través del producto tensorial de Kronecker:

$$H_2 = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad H_{2^n} = H_2 \otimes H_{2^{n-1}}$$

La exigencia de usar la matriz de Sylvester radica en que su estructura indexada refleja perfectamente el espacio vectorial $\mathbb{F}_2^n$. Si se utilizara cualquier otra matriz de Hadamard isomórfica (generada por métodos de Paley, Williamson o secuencias aleatorias), el espectro resultante mostraría comportamientos no aleatorios y sesgos estructurales que destruirían el análisis, ya que las filas de la matriz no se corresponderían con las formas lineales canónicas del espacio de entrada.

Al aplicar la Transformada de Walsh-Hadamard de Sylvester, obtenemos un perfil que mide la correlación exacta entre nuestra función booleana y todas las funciones afines posibles. Por el Teorema de Parseval, la suma de los cuadrados de estos coeficientes de Walsh es constante. Cuando una función minimiza esta correlación de manera uniforme, su espectro de Walsh es completamente plano, tomando únicamente los valores límite $\pm 2^{n/2}$. A estas funciones de máxima no linealidad se les denomina *funciones Bent* [Pommerening \[2014\]](#); [Rothaus \[1976\]](#).

La búsqueda de Pares de Legendre está profundamente anclada a este fenómeno algebraico. La planitud espectral en el dominio booleano (funciones Bent) es el análogo directo de la condición de Densidad Espectral de Potencia (PSD) constante que nuestro software verifica algorítmicamente. Encontrar soluciones óptimas en la teoría de Legendre proporciona vectores base directos para sintetizar funciones con inmunidad teórica máxima frente al criptoanálisis.

1.3. El reto computacional y la combinatoria explosiva

La búsqueda de pares de Legendre es, fundamentalmente, un problema de optimización combinatoria en un espacio de búsqueda exponencial. Dado un par de secuencias binarias (A, B) de longitud ℓ , donde cada elemento $a_i, b_i \in \{-1, 1\}$, el espacio total de combinaciones posibles viene dado por $2^\ell \times 2^\ell = 2^{2\ell}$.

Para una longitud objetivo moderada como $\ell = 75$, esto implica un espacio de 2^{150} candidatos potenciales. Para poner esta cifra en perspectiva, $2^{150} \approx 1,4 \times 10^{45}$, un número astronómico que supera con creces la capacidad de cómputo acumulada de toda la humanidad. Incluso utilizando el superordenador más potente del mundo actual (capaz de realizar exaflops, 10^{18} operaciones por

segundo), la verificación por fuerza bruta tardaría una cantidad de tiempo inabarcable, superior a la edad del universo. **[Comentario: Podrías citar aquí los ordenadores más potentes del mundo y sus capacidades de cómputo para dar una idea concreta.]**

Para comprender la magnitud de este espacio, podemos recurrir a una analogía: si cada candidato generado fuera evaluado en el tiempo que tarda la luz en recorrer un átomo, el proceso seguiría tardando billones de años. En problemas con complejidad temporal exponencial, $\mathcal{O}(2^N)$, el paradigma tradicional de *escalado vertical* (adquirir procesadores más rápidos) resulta fútil. Por ejemplo, duplicar la potencia computacional mundial solo permitiría resolver una secuencia de longitud $\ell + 1$ en el mismo tiempo que ahora toma resolver la longitud ℓ . Por lo tanto, el obstáculo no es meramente tecnológico, sino fundamentalmente algorítmico, lo que justifica y exige la reingeniería y optimización profunda propuesta en este trabajo.

1.3.1. Fundamentos Matemáticos del Método de Compresión

Para abordar este problema intratable, es imperativo recurrir a métodos algebraicos que reduzcan la dimensionalidad del espacio. El *método de compresión* (compression method) es una de las técnicas que se utilizan en este proyecto. Esta técnica proyecta las secuencias originales de longitud ℓ sobre un subgrupo de tamaño menor dividiendo la longitud por un factor d (en este trabajo, $d = 3$, dado que nos centramos en longitudes $\ell \equiv 0 \pmod{3}$).

Sea $A = (a_0, a_1, \dots, a_{\ell-1})$ una secuencia binaria donde cada $a_i \in \{+1, -1\}$. Definimos la secuencia comprimida $A^{(d)}$ de longitud $m = \ell/d$ mediante la suma de los elementos de la secuencia original separados por un salto de m posiciones. Formalmente, el i -ésimo elemento de la secuencia comprimida se calcula como:

$$A_i^{(d)} = \sum_{j=0}^{d-1} a_{i+j \cdot m}, \quad \text{para } i = 0, 1, \dots, m-1$$

Dado que estamos sumando $d = 3$ elementos cuyos valores solo pueden ser $+1$ o -1 , el alfabeto de la nueva secuencia comprimida deja de ser estrictamente binario. Los posibles valores resultantes para cada $A_i^{(3)}$ pertenecen al conjunto finito $\{-3, -1, +1, +3\}$.

Este mapeo reduce drásticamente el espacio de búsqueda. Si determinamos que no existen secuencias comprimidas en este espacio de longitud m que cumplan las condiciones necesarias, se garantiza matemáticamente que tampoco existirán soluciones en el espacio original de longitud ℓ .

Ejemplo 1: Compresión ilustrativa a pequeña escala ($\ell = 9, d = 3$)

Para visualizar el mecanismo, imaginemos una secuencia arbitraria muy pequeña de longitud $\ell = 9$. El factor de compresión es $d = 3$, por lo que la secuencia comprimida tendrá una longitud $m = 9/3 = 3$.

Sea la secuencia original:

$$A = (+1, +1, -1, -1, +1, +1, -1, -1, -1)$$

Aplicando la fórmula, sumamos elementos separados por un salto de $m = 3$ posiciones:

- $A_0^{(3)} = a_0 + a_3 + a_6 = (+1) + (-1) + (-1) = -1$
- $A_1^{(3)} = a_1 + a_4 + a_7 = (+1) + (+1) + (-1) = +1$
- $A_2^{(3)} = a_2 + a_5 + a_8 = (-1) + (+1) + (-1) = -1$

La secuencia resultante comprimida es $A^{(3)} = (-1, +1, -1)$. Se ha pasado de evaluar 9 variables binarias a evaluar un array de tan solo 3 variables enteras.

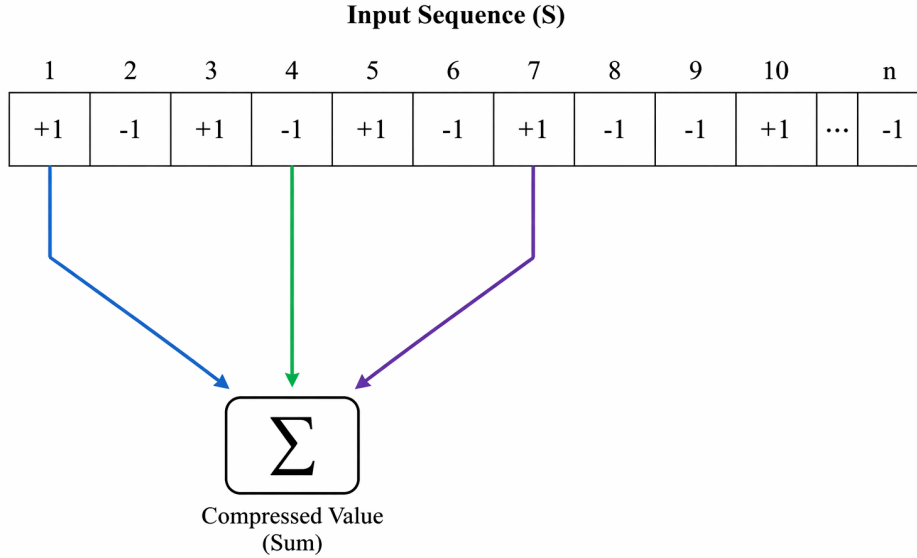


Ilustración 2: Representación visual del mapeo de compresión por un factor $d=3$. Los elementos binarios colapsan algebraicamente mediante sumas periódicas en un único elemento entero, reduciendo la dimensionalidad del problema original.

Ejemplo 2: El caso de estudio principal ($\ell = 75, d = 3$)

Extrapolando este concepto a nuestro objetivo principal en el supercomputador, partimos de secuencias desconocidas A y B de longitud $\ell = 75$. Aplicando $d = 3$, obtenemos vectores comprimidos $A^{(3)}$ y $B^{(3)}$ de longitud $m = 25$.

Esta compresión tiene una propiedad espectral fundamental: la densidad espectral de potencia (PSD) de las secuencias comprimidas preserva una relación proporcional con la secuencia original en las frecuencias correspondientes. Por tanto, el software no necesita buscar en el inabarcable espacio de 2^{150} combinaciones originales, sino que evalúa candidatos en el espacio de combinaciones de 25 elementos sobre el alfabeto $\{-3, -1, +1, +3\}$.

Una vez que el algoritmo detecta un par comprimido ($A^{(3)}, B^{(3)}$) cuya PSD en el dominio frecuencial es compatible con la constante objetivo $2\ell+2 = 152$, sabemos que es un candidato viable, y procedemos a su descompresión para verificar el Par de Legendre exacto.

1.4. Propiedades matemáticas fundamentales

Para comprender la lógica implementada en el código desarrollado, es necesario formalizar las propiedades que nuestro algoritmo verifica. Un par de Legendre de longitud ℓ está formado por dos secuencias $A = \{a_i\}$ y $B = \{b_i\}$ con valores en $\{-1, +1\}$. **[Comentario: Aquí se repite información, que ya se ha dado en otro sitio. La definición de estas secuencias ya está dada en la sección de antecedentes, pero aquí se puede profundizar un poco más en la explicación de cada propiedad.]**

1.4.1. Función de Autocorrelación Periódica (PAF)

La condición en el dominio del tiempo exige que la suma de las autocorrelaciones de A y B sea una función delta (cero en todo punto salvo en el origen):

$$\text{PAF}_A(k) + \text{PAF}_B(k) = \sum_{i=0}^{\ell-1} (a_i a_{i+k} + b_i b_{i+k}) = -2, \quad \forall k \neq 0 \quad (\text{mód } \ell)$$

1.4.2. Densidad Espectral de Potencia (PSD)

Verificar la PAF tiene una complejidad computacional de $O(\ell^2)$. Para acelerar el cálculo, trasladamos el problema al dominio de la frecuencia usando la Transformada Discreta de Fourier (DFT). La condición equivalente, y la que nuestro software verifica, es que la suma de las energías espectrales sea constante:

$$|\text{DFT}_A(k)|^2 + |\text{DFT}_B(k)|^2 = 2\ell + 2, \quad \forall k$$

Esta propiedad permite una verificación en tiempo $O(\ell)$ una vez calculada la DFT, siendo mucho más eficiente para el filtrado masivo de millones de candidatos por segundo.

[Comentario: No defines que es el dominio frecuencia ni la DFT.] La formulación en el dominio frecuencial establece que un par es válido si y solo si la suma de sus densidades espectrales es exactamente igual a la constante $2\ell + 2$ para cualquier frecuencia discreta k . Por ende, cualquier mínima desviación de este valor en una sola frecuencia descarta inmediatamente al candidato. Dado que esta evaluación debe realizarse para cada una de las trillones de combinaciones generadas, el cálculo repetitivo de las multiplicaciones complejas de la DFT se erige como el principal "cuello de botella" computacional del sistema. Optimizar el flujo de instrucciones de esta métrica es el objetivo primordial de la metaprogramación desarrollada.

1.5. Objetivos y metodología de desarrollo

Para abordar un problema de esta envergadura computacional, fue necesario establecer un marco metodológico estricto y seleccionar cuidadosamente la pila tecnológica (*tech stack*).

1.5.1. Metodología en Cascada (Waterfall)

A diferencia de proyectos de software comercial donde los requisitos cambian constantemente (lo que favorecería metodologías Ágiles), este problema matemático está rígidamente definido desde el principio. Por ello, se optó por un enfoque en **Cascada**.

Cada fase de desarrollo se concibió para identificar y solucionar el cuello de botella de la fase anterior:

1. **Fase Matemática:** Definición teórica del problema y reducción algebraica mediante cosets ciclotómicos.
2. **Fase de Prototipado Secuencial:** Implementación de algoritmos base en C que escriben/leen en disco para validar la correctitud.
3. **Fase de Refinamiento Algorítmico:** Transición a algoritmos de Búsqueda en Profundidad (DFS) en memoria con estrategias de poda (pruning) para evitar el colapso por Entrada/Salida (I/O).
4. **Fase de Paralelización y Metaprogramación:** Generación de código a medida y despliegue masivo (OpenMP/Slurm) para romper los límites de tiempo del superordenador.

[Comentario: No se si es necesario poner algo más detallado de las fases.]

1.5.2. Herramientas y Decisiones Arquitectónicas

El entorno de ejecución fue el Supercomputador Altamira (IFCA). Las herramientas seleccionadas fueron:

- **Lenguajes C y C++:** Elegidos por su nula sobrecarga en tiempo de ejecución, acceso directo a memoria y madurez de sus compiladores para vectorización.
- **GCC (GNU Compiler Collection):** Utilizando las *flags* `-O3 -march=native` para obligar al compilador a usar las instrucciones SIMD específicas de los procesadores del clúster.

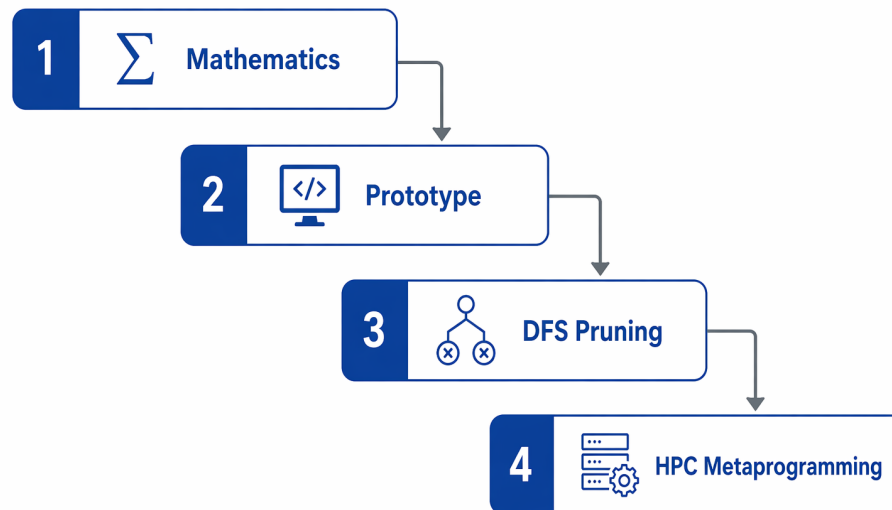


Ilustración 3: Metodología de desarrollo secuencial (Cascada). Cada etapa del proyecto fue diseñada iterativamente para aislar, analizar y eliminar los cuellos de botella computacionales identificados en la fase previa.

- **OpenMP vs. MPI:** Se evaluó el uso de MPI (Message Passing Interface) para paralelizar la búsqueda entre múltiples nodos. Sin embargo, **se descartó a favor de OpenMP + Slurm Arrays**. El problema de búsqueda de Legendre es un problema *embarrassingly parallel* (vergonzosamente paralelo) sin necesidad de que los nodos se comuniquen entre sí. MPI habría añadido una complejidad innecesaria y un sobrecoste de comunicación (*network overhead*) contraproducente. La paralelización híbrida (OpenMP para hilos dentro de un nodo, y Slurm para distribuir entre nodos de forma estanca) resultó ser la aproximación óptima.
- **Sistema de Colas Slurm:** Fundamental para la gestión de trabajos mediante *Job Arrays*, eludiendo los límites de tiempo de la plataforma [Yoo et al. \[2003\]](#).

1.5.3. Estructura de la Memoria

[Comentario: Aquí estaría bien una sección de la estructura de la memoria, que se va a decir en cada capítulo.]

2 Desarrollo e Innovaciones Algorítmicas

[Comentario: Esto parece que contradice un poco lo que has dicho antes de que el desarrollo se hizo en cascada. Aquí parece que se hizo de forma iterativa. Dale una vuelta a esto o cambia la metodología.] El núcleo de este Trabajo de Fin de Grado reside en la paulatina evolución de la arquitectura del software, concebida para doblegar la intratabilidad computacional de los espacios de búsqueda masivos asociados a los pares de Legendre. Este capítulo traza la hoja de ruta integral del proyecto, detallando el progreso secuencial desde la formulación de los algoritmos matemáticos iniciales hasta el despliegue de un generador de código de alto rendimiento en la infraestructura del supercomputador Altamira.

La búsqueda de pares de Legendre donde el número de posibilidades crece de forma exponencial con ℓ no puede abordarse mediante un único incremento de fuerza bruta, sino que requiere una orquestación de técnicas transversales en múltiples niveles de abstracción informática y matemática. En primer lugar, se expondrá la base teórica del tamaño del espacio de búsqueda mediante órbitas de equivalencia ciclotómica, estableciendo los principios algebraicos sobre los cuales operan todos los algoritmos posteriores.

[Comentario: La forma de expresarse es un poco confusa, por ejemplo, no utilizar asfixiado, sino que los límites] A continuación, se documentará la evolución iterativa del motor de búsqueda. Se analizará el fracaso de las primeras implementaciones, cuya aplicabilidad se restringía por los severos cuellos de botella de entrada y salida (I/O) en el disco físico, y se justificará la transición hacia un modelo dinámico de Búsqueda en Profundidad (DFS) ejecutado íntegramente en la memoria principal. En este punto, se detallará cómo la inyección de heurísticas de poda matemática y el rediseño de las estructuras de datos para aprovechar operaciones a nivel de bit (paralelismo SWAR intra-registro) permitieron al software descartar billones de ramas estériles sin la necesidad de evaluarlas computacionalmente.

[Comentario: Se repiten muchas cosas, me gustaría que se explicara más] Aun con estas mejoras, la optimización algorítmica clásica está muy limitada por el tamaño del espacio de búsqueda. Por ello, la segunda mitad del capítulo se adentra en el cambio de paradigma hacia la computación distribuida y la persistencia de datos probabilística. Se expondrá la integración de Filtros de Bloom para orquestar una estrategia *Meet-in-the-Middle* capaz de alterar la complejidad temporal del problema aislando sus subespacios, logrando contener el colapso teórico de la memoria RAM.

Finalmente, se coronará el desarrollo detallando la arquitectura de metaprogramación: la creación de un sistema matriz capaz de transcribir su propio código fuente en C++, desenrollando bucles y eliminando la latencia de las ramificaciones condicionales para acoplarse de forma nativa a las instrucciones vectoriales del procesador anfitrión. Este recorrido ilustra, paso a paso, la metamorfosis de un problema analítico de matemáticas discretas en un desafío integral de ingeniería de software, arquitecturas superescalares y orquestación de clústers de supercomputación.

2.1. Reducción Algebraica: Cosets Ciclotómicos

En lugar de iterar por cada bit individual de una secuencia de longitud ℓ , las propiedades de simetría de las secuencias de Legendre permiten agrupar los índices en clases de equivalencia bajo multiplicación por 2, denominadas **Cosets Ciclotómicos**. Formalmente, para un índice $s \in \{0, 1, \dots, \ell - 1\}$, el coset ciclotómico C_s módulo ℓ se define como la órbita generada al multiplicar iterativamente por 2:

$$C_s = \{s \cdot 2^i \pmod{\ell} \mid i \in \mathbb{N}\}$$

El tamaño de cada órbita está determinado por el orden multiplicativo de 2 módulo $\ell/\text{mcd}(s, \ell)$. Esta operación matemática divide el dominio discreto original en conjuntos completamente disjuntos. La principal restricción algebraica de este método dicta que, si la secuencia $(a_0, a_1, \dots, a_{\ell-1})$ es un candidato válido, entonces para cualquier par de índices $j, k \in C_s$, se cumple de manera estricta que $a_j = a_k$. En otras palabras, la secuencia es invariante ante una *decimación diádica*. Esta propiedad **[Comentario: No es que colapsa la dimensionalidad, prefiero decir que se reduce el espacio de búsqueda.]** reduce el número de elementos de la secuencia independientes: ya no decidimos el valor de ℓ variables independientes, sino el valor de un número sustancialmente menor de cosets, actuando cada uno como un bloque atómico.

Para el caso objetivo $\ell = 75$, la partición del anillo $\mathbb{Z}_{75}$ genera exactamente 45 cosets distintos, cada uno con una cardinalidad que varía dependiendo de la estructura multiplicativa del grupo subyacente (en este caso, tamaños típicos de 1, 2, 4 o 20 elementos). Matemáticamente, esto significa que el espacio de búsqueda original de 2^{75} combinaciones se proyecta sobre un subespacio isomórfico de 2^{45} . Esta reducción matemática elimina redundancias intrínsecas y reduce el exponente de complejidad en 30 magnitudes completas. **[Comentario: Explica mejor el exponente de complejidad, porque no está claro]**

[Comentario: Ahora haces un salto muy grande, porque antes hemos dicho que el espacio de búsqueda era $2^{2\ell}$] A pesar del extraordinario recorte dimensional, evaluar empíricamente 2^{45} candidatos (aproximadamente $3,5 \times 10^{13}$ evaluaciones) sigue siendo un reto masivo que desborda las capacidades de la fuerza bruta lineal. Para ilustrar la dificultad, a una velocidad sostenida de 100,000 iteraciones por segundo por núcleo computacional, verificar el espacio reducido consumiría más de 11 años ininterrumpidos en hardware estándar, lo cual exige obligatoriamente la transición hacia estrategias algorítmicas avanzadas y podas.

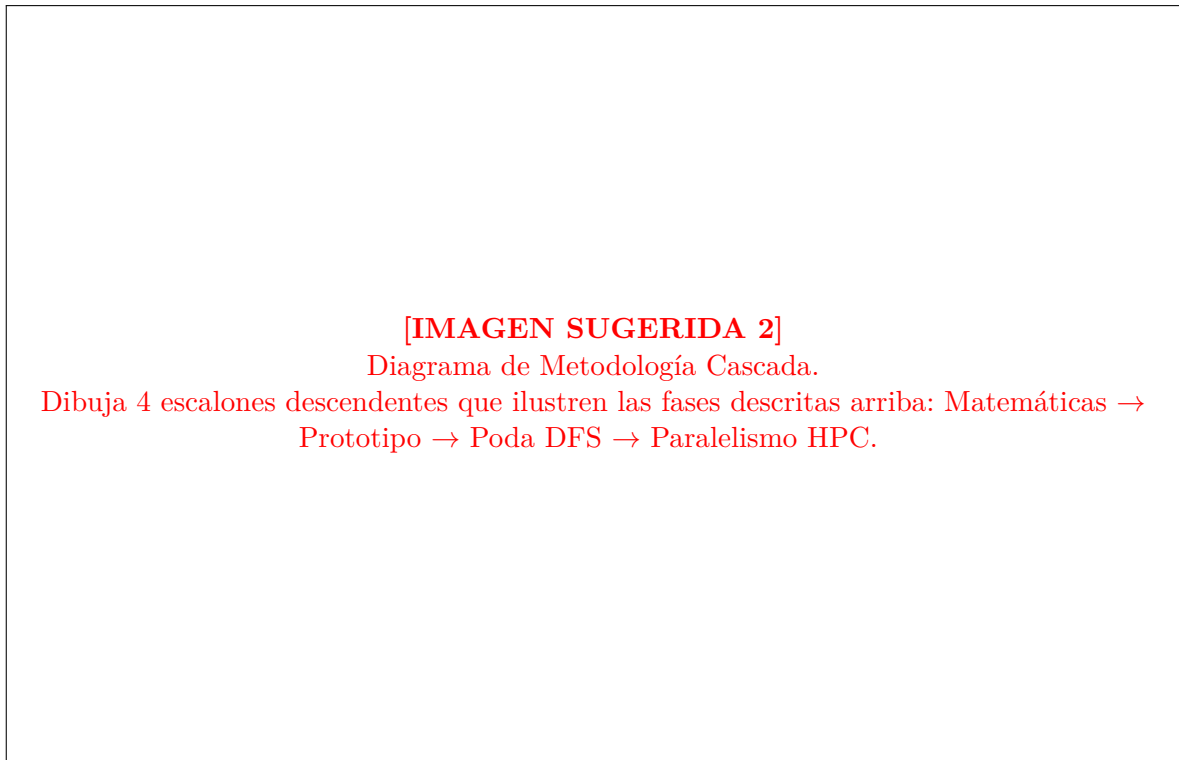


Ilustración 4: Metodología de desarrollo en Cascada orientada a la eliminación progresiva de cuellos de botella.

2.1.1. Herramientas y Decisiones Arquitectónicas

El entorno de ejecución fue el Supercomputador Altamira (IFCA). Las herramientas seleccionadas fueron:

- **Lenguajes C y C++:** Elegidos por su nula sobrecarga en tiempo de ejecución, acceso directo a memoria y madurez de sus compiladores para vectorización.
- **GCC (GNU Compiler Collection):** Utilizando las *flags* `-O3 -march=native` para obligar al compilador a usar las instrucciones SIMD específicas de los procesadores del clúster.
- **OpenMP vs. MPI:** Se evaluó el uso de MPI (Message Passing Interface) para paralelizar la búsqueda entre múltiples nodos. Sin embargo, **se descartó a favor de OpenMP + Slurm Arrays**. El problema de búsqueda de Legendre es un problema *embarrassingly parallel* (vergonzosamente paralelo) sin necesidad de que los nodos se comuniquen entre sí. MPI habría añadido una complejidad innecesaria y un sobrecoste de comunicación (*network overhead*) contraproducente. La paralelización híbrida (OpenMP para hilos dentro de un nodo, y Slurm para distribuir entre nodos de forma estanca) resultó ser la aproximación óptima.
- **Sistema de Colas Slurm:** Fundamental para la gestión de trabajos mediante *Job Arrays*, eludiendo los límites de tiempo de la plataforma.

3 Desarrollo e Innovaciones Algorítmicas

El núcleo de este TFG reside en la paulatina evolución del software para lograr verificar espacios de búsqueda masivos. Este capítulo detalla el progreso desde los algoritmos matemáticos iniciales hasta el despliegue del generador optimizado.

3.1. Reducción Algebraica: Cosets Ciclotómicos

En lugar de iterar por cada bit individual de una secuencia de longitud N , las propiedades matemáticas de las secuencias de Legendre permiten agrupar los índices en conjuntos invariantes llamados **Cosets Ciclotómicos**.

Si un bit cambia en un índice perteneciente a un coset, todos los demás bits de ese mismo coset deben obligatoriamente tener el mismo valor. Esto reduce drásticamente las variables de decisión. Por ejemplo, para $N = 75$, la longitud se descompone en 45 cosets distintos. Esto significa que el espacio de búsqueda baja de un inexplorable 2^{75} a un más tratable 2^{45} .

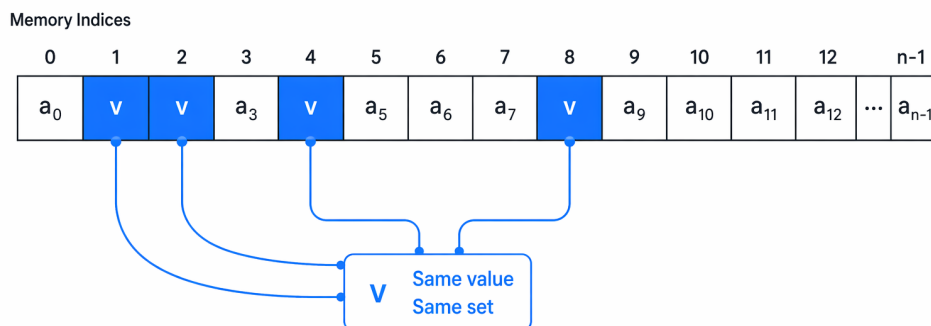


Ilustración 5: Agrupación de índices mediante cosets ciclotómicos. La restricción algebraica que obliga a múltiples posiciones del array a compartir el mismo valor reduce drásticamente la entropía del espacio de búsqueda.

3.2. Evolución del Software: De la I/O a la Búsqueda en Profundidad

El desarrollo se estructuró en versiones progresivas del código (`main_txt.c`, `main-1.c`, `main.c`), cada una superando empírica y arquitectónicamente las deficiencias y cuellos de botella aislados en la anterior.

3.2.1. Fase 1: Fuerza Bruta sobre Cosets y Archivos Intermedios

El código inicial (`main_txt.c`) operaba bajo una arquitectura estrictamente secuencial y altamente acoplada al sistema de archivos. Aplicaba el concepto de los cosets generando exhaustivamente combinaciones mediante bucles iterativos, calculando sus espectros, y escribiendo la totalidad de los datos resultantes en archivos de texto plano masivos (ej. `comp_dft_cosets.txt`).

Posteriormente, la función `buscar_pares_complementarios` actuaba como una segunda etapa aislada. Leía las PSDs almacenadas en disco y ejecutaba un recorrido de orden cuadrático comparando cada registro para buscar combinaciones que sumaran el objetivo exacto ($2N + 2$). Finalmente, se empleaba la función `find_matches_files` para cruzar los archivos resultantes y mapear las posiciones con las secuencias originales correspondientes.

```

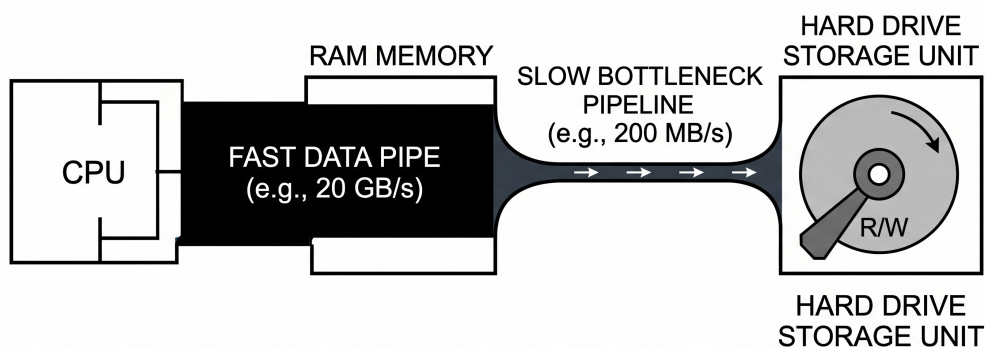
1 for (size_t i = 0; i < total; i++) {
2     for (size_t j = i + 1; j < total; j++) {
3         if ((coll[i] + coll[j]) == objetivo) {
4             fprintf(f_out, "%s\n%s\n\n", secuencias[i], secuencias[j]);
5             encontrados++;
6         }
7     }
8 }

```

Listing 3.1: Fragmento de la lógica inicial de cruce de ficheros

El problema: Aunque matemáticamente correcto, este enfoque colapsaba rápidamente al escalar la dimensión de ℓ . La experimentación demostró que el cuello de botella crítico no residía en la capacidad de la Unidad Central de Procesamiento (CPU) para resolver la Transformada de Fourier, sino en la limitación física de **Entrada/Salida (I/O) del disco duro**. Escribir y leer iterativamente cadenas de caracteres desencadenaba un fenómeno de latencia extrema (*I/O Wait*), saturando el ancho de banda del bus SATA o PCIe. Además, la carga térmica y de llamadas al sistema (*syscalls* como `fprintf`) hacían imposible explorar secuencias largas en tiempos computacionales razonables.

COMPUTER ARCHITECTURE I/O BOTTLENECK DIAGRAM



THE I/O BOTTLENECK: FAST CPU and RAM are RESTRICTED by
a SLOW HARD DRIVE INTERFACE.

Ilustración 6: Esquema del cuello de botella de Entrada/Salida (I/O). El rendimiento algorítmico de la Fase 1 se vio severamente estrangulado por la latencia del bus de almacenamiento físico, forzando la migración a un modelo in-memory.

3.2.2. Fase 2: Búsqueda en Profundidad (DFS) y Podas (Pruning)

Para solucionar el colapso del disco y suprimir por completo la dependencia del subsistema I/O, el código evolucionó a `main-1.c`, abandonando la escritura masiva a favor de un algoritmo algorítmico.

camente más inteligente: una Búsqueda en Profundidad (DFS) ejecutada íntegramente en memoria RAM.

En lugar de generar todas las combinaciones posibles en bloque (2^{45} conjuntos estáticos) y evaluarlas de forma diferida, el algoritmo construye la secuencia elemento a elemento, descendiendo secuencialmente por niveles del árbol DFS asimilado a los índices de los cosets. La innovación crítica en esta fase es la introducción del mecanismo dinámico de **Podas (Bounds)**.

Antes de descender al siguiente nivel del árbol de exploración para asignar un estado definitivo a un coset, el algoritmo ejecuta un subrutina de acotación matemática (`check_bound`). Esta función proyecta el peor escenario espectral posible para la secuencia parcial construida hasta el momento. Utilizando la linealidad de la Transformada Discreta de Fourier, calculamos la cota inferior de energía acumulada y la restamos del objetivo global $2\ell + 2$. La inecuación base que guía la poda exige que:

$$\max_k (|\text{DFT}_{\text{parcial}}(k)| - \Delta_{\text{restante}})^2 \leq 2\ell + 2$$

Si la discrepancia espectral en cualquier frecuencia discreta k supera la energía máxima que los cosets restantes podrían teóricamente aportar (Δ_{restante}), el teorema de Parseval nos garantiza axiomáticamente que es imposible que la rama en curso derive en un par válido, sin importar qué valores tomen los nodos inferiores del árbol. En ese instante, el algoritmo aborta la recursión y poda el subárbol íntegro, eludiendo la evaluación de millones de nodos estériles.

La arquitectura interna que gobierna esta lógica se formaliza en el siguiente pseudocódigo estructural, representativo del motor lógico ejecutado en `main.c`:

```

1 void dfs_explore(DFSContext *ctx, int coset_index) {
2     if (coset_index == ctx->num_cosets) {
3         verify_and_store_candidate(ctx);
4         return;
5     }
6
7     if (!is_mathematically_viable(ctx)) {
8         return;
9     }
10
11     uint64_t valid_states = retrieve_valid_bitmask(ctx, coset_index);
12
13     if (valid_states == 0) {
14         return;
15     }
16
17     while (valid_states > 0) {
18         int state = __builtin_ctzll(valid_states);
19
20         apply_state_to_context(ctx, coset_index, state);
21
22         dfs_explore(ctx, coset_index + 1);
23
24         revert_state_from_context(ctx, coset_index, state);
25
26         valid_states &= (valid_states - 1);
27     }
28 }
```

Listing 3.2: Algoritmo central de Búsqueda en Profundidad (DFS) con poda dinámica orientada a bits

Esta técnica recursiva transformó un problema de almacenamiento inabarcable en un proceso dinámico computacional, limitando el volumen del espacio de búsqueda exclusivamente a las regiones matemáticamente probables y reduciendo la carga computacional en varios órdenes de magnitud reales.

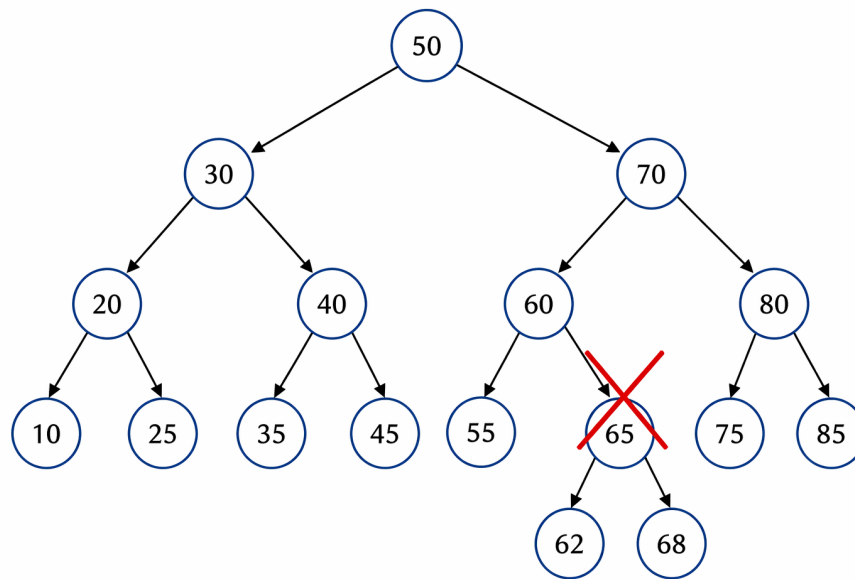


Ilustración 7: Esquema de Búsqueda en Profundidad (DFS) con poda (pruning) heurística. La evaluación anticipada de los límites espectrales inferiores permite descartar ramas completas del árbol de combinaciones sin llegar a explorarlas.

3.2.3. Fase 3: El Estado del Arte (Bitsets y Ordenación Heurística)

La versión definitiva programada puramente en C (`main.c`) refinó el núcleo del motor DFS para operar bajo principios rigurosos de Arquitectura y Alto Rendimiento de computadores, explotando la jerarquía de memoria de la CPU y los registros internos vectoriales.

1. Estructuras Bitset y paralelismo intra-registro (SWAR): Tradicionalmente, recorrer arrays multidimensionales en C implica iteraciones `for` que acarrearán costosas penalizaciones por predicciones de salto fallidas y subutilización del ancho de banda de la memoria de caché L1. En vez de ello, se implementaron matrices compactas de bits (`ge_bitsets`, `le_bitsets`). Estas estructuras representan una técnica conocida como SWAR (*SIMD Within A Register*), que permite comprimir los estados lógicos de los candidatos dentro de enteros nativos de 64 bits.

De esta manera, el filtrado permite evaluar 64 posibles estados de validación simultáneamente mediante un único ciclo lógico de instrucciones lógicas booleanas (AND, OR) en la Unidad Aritmético Lógica (ALU). Posteriormente, la instrucción en hardware `__builtin_popcountll`, que extrae la cardinalidad de bits activos en una sola operación de máquina, verifica la existencia de candidatos superando las trabas de ramificación de código.

```

1 int count = 0;
2 for (size_t w = 0; w <= last_word; w++) {
3     uint64_t bits = ge[j][l][w] & le[j][u][w];
4     count += __builtin_popcountll((unsigned long long)bits);
5 }
6 if (count == 0) return false;

```

Listing 3.3: Filtrado masivo mediante operaciones de bits (SWAR) y hardware POPCNT

2. Reordenación Heurística de Cosets: El factor crítico de eficiencia en cualquier algoritmo probabilístico de ramificación y poda (Branch and Bound o DFS puro) recae en la asimetría del árbol. El lema subyacente es la necesidad de “equivocarse rápido”. Si podemos ramificar infructuosas en el nivel de profundidad 2, economizamos colosalmente más tiempo de CPU que si la invariante crítica falla en el nivel 40.

Para lograr esto, la subrutina `reorder_cosets_by_candidate_pairs` efectúa un análisis a priori.

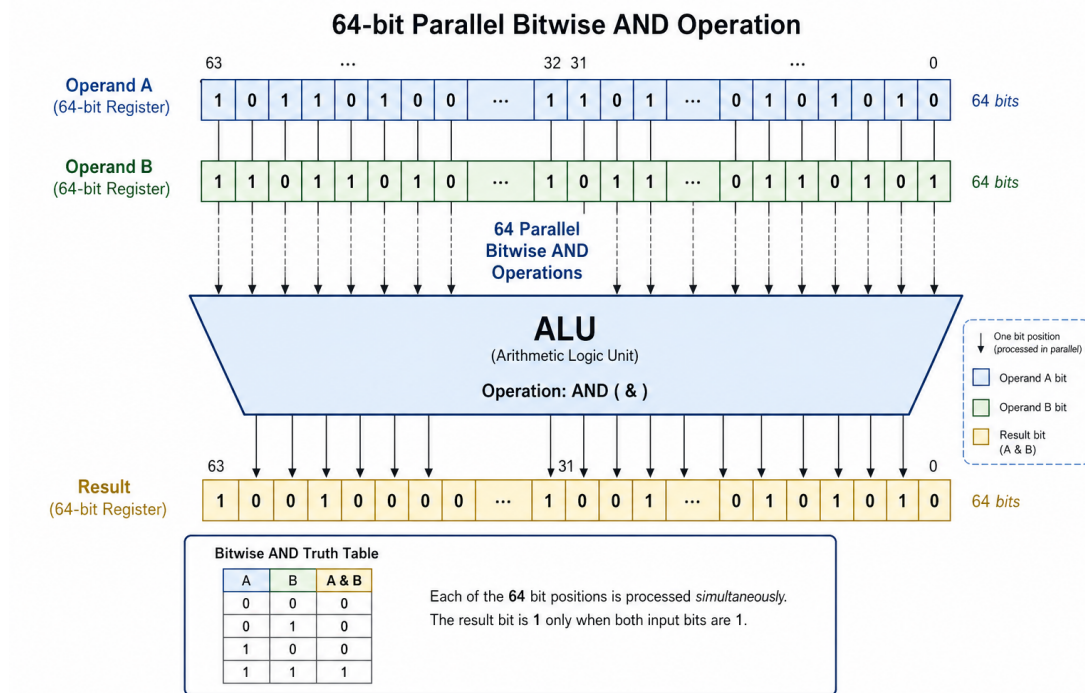


Ilustración 8: Paralelismo intra-registro (SWAR). Al compactar los estados de viabilidad en enteros de 64 bits, la ALU del procesador puede aplicar validaciones booleanas masivas en un único ciclo de reloj, eludiendo saltos condicionales.

Observa la varianza y la restricción teórica de peso que impone cada bloque coset, reestructurando el árbol topológico dinámicamente. Al evaluar las ramificaciones más excluyentes en la cúspide de la recursión, la eficiencia algorítmica de la poda se maximiza sustancialmente, conteniendo la explosión combinatoria basal.

3.3. Reducción de Complejidad Extrema: Filtros de Bloom y Estrategia Meet-in-the-Middle

[Comentario: Intenta no utilizar palabras como “inmensos”, “colossalmente”, “extrema”, etc. que no aportan información técnica.] A pesar de los avances obtenidos integrando lógica DFS y podas por Bitsets, el crecimiento exponencial del espacio muestral en longitudes $\ell > 70$ dictaminaba la necesidad de un cambio de paradigma para asimilar de forma paralela y estanca ambos subespacios de búsqueda correspondientes a las secuencias gemelas A y B . El problema original obliga a evaluar cruzadamente los candidatos de A contra los candidatos de B en bucles condicionales anidados, produciendo una complejidad de cruce cuadrática de la clase $\mathcal{O}(N_A \times N_B)$.

Sabemos que la condición terminal de éxito es una sumatoria aditiva:

$$PSD_A(s) + PSD_B(s) = 2\ell + 2$$

Algebraicamente, esto significa que, si fijamos unilateralmente la distribución espectral de una secuencia candidata A , sabemos de antemano exactamente qué PSD complementaria es exigida rigurosamente para acoplarse y conformar un par de Legendre válido:

$$PSD_{B_buscado}(s) = 2\ell + 2 - PSD_A(s)$$

Siguiendo con este razonamiento, si pudiéramos guardar en un diccionario todas las proyecciones espectrales deseadas e iteradas por A , la búsqueda recaería en calcular B una sola vez y verificar en

tiempo constante si su espectro coincide en el registro enlazado de memoria. Esto cristaliza la estrategia denominada **Meet-in-the-Middle**, la cual altera asintóticamente la complejidad subyacente a $\mathcal{O}(N_A + N_B)$. Sin embargo, una base de datos tabular o tabla hash que almacene trillones de vectores integrales multidimensionales superaría holgadamente los Terabytes de volumen, colapsando instantáneamente la memoria RAM, los buses de transferencia inter-nodos y los niveles de swap virtual de cualquier componente físico dentro de la supercomputadora Altamira.

La solución de ingeniería arquitectónica implementada para salvar esta barrera termodinámica y estructural es la inserción de un **Filtro de Bloom** presentado en Bloom [1970].

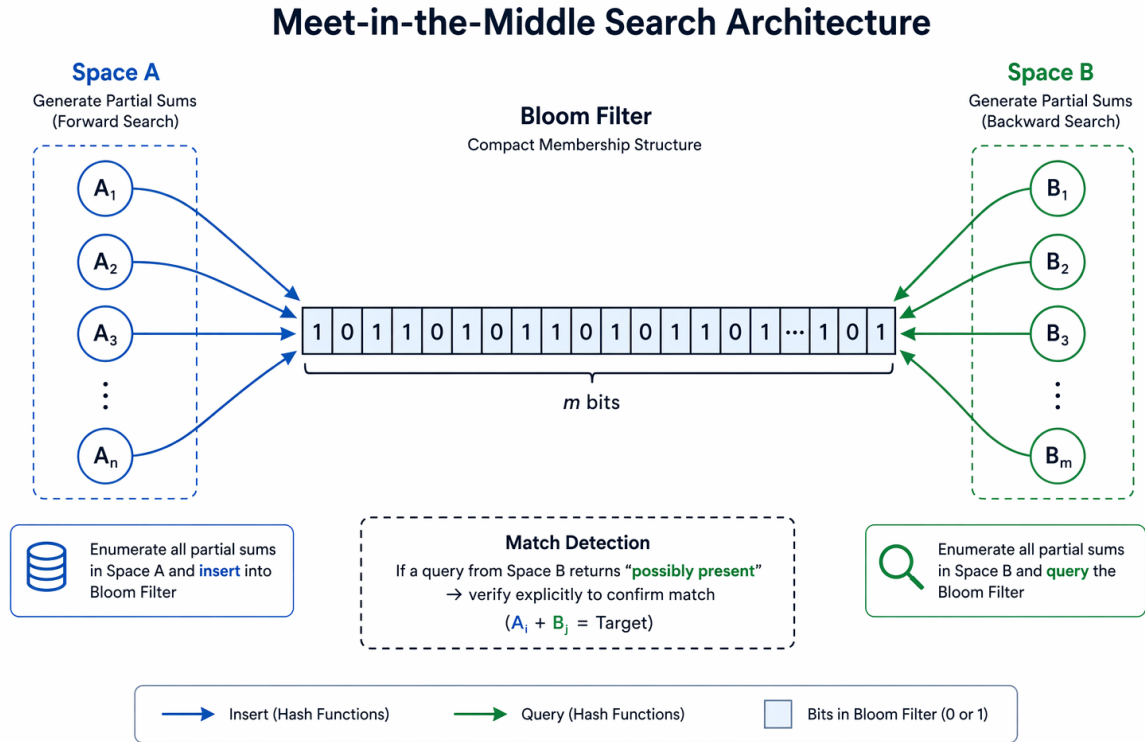


Ilustración 9: Estrategia Meet-in-the-Middle orquestada mediante un Filtro de Bloom. Esta estructura de datos actúa como una memoria probabilística ultrarrápida que cruza el espacio de secuencias A con el espacio B sin saturar la memoria RAM.

[Comentario: ¿Qué es hermeticamente iniciable?] Un Filtro de Bloom es una estructura de datos probabilística altamente comprimida soportada por un vector contiguo de bits de tamaño m , inicializado a ceros, y un conjunto algorítmico de k funciones de proyección independientes o *hashes*. Cuando procesamos iterativamente la Fase 1 del algoritmo superior, cada perfil espectral válido (PSD_A) que elude la poda DFS se somete a k transformaciones *hash* unívocas, utilizando semillas de derivación iterativas basadas en el algoritmo multiplicativo **[Comentario: Utiliza referencias, no lo pongas a fuego]** FNV-1a. Los resultados numéricos generados se modulan para ajustarse al rango métrico de memoria y proyectan el perfil matemático subyacente sobre k índices del vector global m , activando sus bits lógicos a 1.

[Comentario: Como estamos utilizando una longitud fija, no menciones asintóticamente, sino que habla de la práctica.] El diseño probabilístico garantiza rigurosamente la **ausencia de falsos negativos**: si durante la Fase 2 una secuencia candidata B comprueba sus propias k posiciones mapeadas en el filtro y halla un solo bit en estado lógico 0, se descarta de forma irrefutable su compatibilidad sin riesgo absoluto de error. El porcentaje estadístico de colisión natural conocido como **falsos positivos** ε , es decir, secuencias B erróneas que coinciden fortuitamente en índices previamente activados por secuencias distintas y deben ser filtradas ex post facto, está gobernado por la

función de tolerancia paramétrica:

$$\varepsilon \approx \left(1 - e^{-k \cdot \frac{n}{m}}\right)^k$$

donde n expresa el volumen total aproximado de perfiles matemáticos singulares insertados en la Fase 1. En nuestro despliegue experimental distribuido en la arquitectura Altamira, se optó por instanciar y dimensionar un vector inmutable de $m = 10^9$ bits (requiriendo apenas 120 megabytes constantes de memoria RAM en el *heap* por cada nodo de cálculo) y $k = 3$ funciones de desbordamiento (hashing) simultáneas. Esta calibración técnica consolidó una tasa experimental de colisiones por falsos positivos virtualmente asintótica al cero, confinando la explosión volumétrica combinatoria sin comprometer la resolución algebraica íntegra.

```

1 inline uint64_t compute_spectrum_hash(int* target_array, int length, uint64_t
  internal_seed) {
2     uint64_t hash_digest = 14695981039346656037ULL + internal_seed;
3     for(int i = 0; i < length; i++) {
4         hash_digest ^= (uint64_t)target_array[i];
5         hash_digest *= 1099511628211ULL;
6     }
7     return hash_digest;
8 }

```

Listing 3.4: Generador de proyecciones Hash paramétricas (FNV-1a) para su uso vectorizado en Filtros de Bloom concurrentes

Esta técnica de compresión de persistencia permitió escalar en HPC. Provocó que los nodos computacionales satélites aislasen las intersecciones del *Meet-in-the-Middle* sin concurrir a bloqueos de memoria distribuida o requerir colas transaccionales de sincronización por interfaz de red.

3.4. Estrategia de Metaprogramación: El Generador en C++

Para aprovechar todas las optimizaciones matemáticas probadas empíricamente, instanciar eficientemente los hilos en paralelo mediante el framework OpenMP, integrar atómicamente los Filtros de Bloom y superar el hándicap estructural derivado de un código interpretativo en la fase de evaluación vectorial, se transicionó a la herramienta arquitectónica definitiva: el **Generador de Buscadores**.

Bajo una arquitectura de metaprogramación explícita, la herramienta principal es una aplicación maestra orquestada en lenguaje C++ que no aplica rutinas para buscar pares de Legendre directamente. Por el contrario, asume el rol de transpilador: lee una longitud teórica unívoca (ℓ) e iterativamente *escribe el código fuente C++* de un buscador puramente dedicado y enlazado estáticamente a esa dimensión.

3.4.1. Cálculo Simbólico y Arquitectura Superescalar (Loop Unrolling)

Los procesadores lógicos modernos que arman los clusters de High Performance Computing (como la familia Intel Xeon o AMD EPYC de Altamira) operan dictados por un enfoque de procesamiento superescalar, donde el sustrato físico de hardware adivina proactivamente las ramificaciones condicionales pre-evaluadas por el compilador y segmenta las operaciones solapadas mediante múltiples *pipelines* de instrucción («ILP», Instruction-Level Parallelism).

Sin embargo, los algoritmos tradicionales de bucles dependientes para funciones complejas como la Transformada Rápida de Fourier evalúan recursivamente sentencias o iteraciones **for**. Esto penaliza el hardware masivamente, rompiendo la fluidez del *pipeline* forzando vaciados iterativos de línea de caché (*cache misses*) toda vez que el algoritmo de predicción de bifurcación erra en anticipar el fin de un bucle de variable paramétrica, o por dependencias indirectas (*loop-carried dependencies*).

El **generador_de_generadores** escrito en entorno C++ sobrepasa holgadamente esta limitación estructural intrínseca del compilador genérico mediante la táctica extrema de **desenrollado de bucles algorítmicos** (*Loop Unrolling*) llevada acabo integralmente a nivel de metaprogramación por

Flujo de Transpilación y Metaprogramación

1. **Parámetro Numérico:** $\ell = 75$.
- ↓
2. **Generación Abstracta:** `generador_de_generadores.cpp` ejecuta cálculos simbólicos complejos de cosenos senos, generando estructuras AST y matrices para un Filtro de Bloom a medida.
- ↓
3. **Emisión de Código:** El generador escupe el volcado a disco del código `compress.75.cpp`, 100% desenrollado y desprovisto de lógica polimórfica (hardcoded).
- ↓
4. **Compilación Dependiente de Hardware:** Interviene `g++ -O3 -march=native`.
- ↓
5. **Despliegue HPC:** Ejecución estanca mediante Lotes e Hilos concurrentes en Slurm.

Ilustración 10: El metaprograma consolida todas las fases de optimización en un flujo automático.

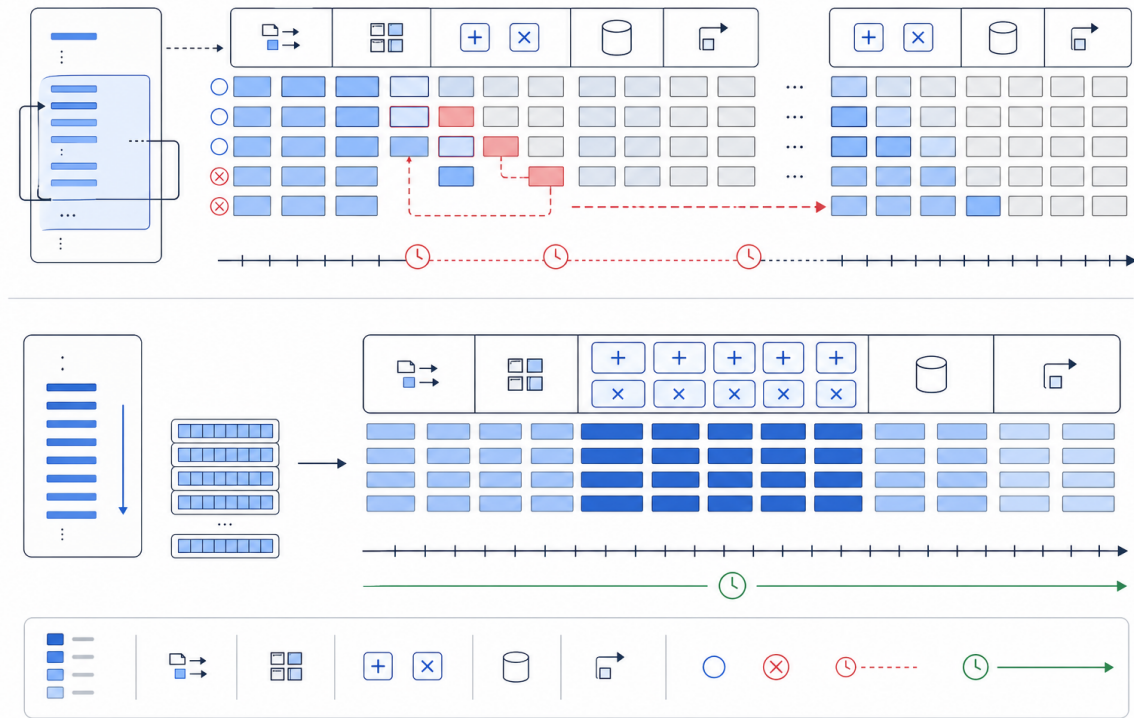


Ilustración 11: Comparativa de ejecución en el cauce (*pipeline*) del procesador. Arriba: Código iterativo tradicional penalizado por fallos en la predicción de saltos. Abajo: Código metaprogramado y desenrollado (*Loop Unrolling*) explotando el paralelismo a nivel de instrucción (ILP) mediante registros SIMD.

abstracción de texto. Durante la fase primaria, previa a la propia existencia del código ejecutable final, este motor genera estáticamente y explora las ramas del Árbol de Sintaxis Abstracta subyacente para desplegar la Transformada Discreta de Fourier de orden dimensional ℓ . Al pre-computar simbólicamente sobre coma flotante las frecuencias analíticas ω^k referidas a las raíces complejas de la unidad y transcribirlas en texto inmutable asimiladas como constantes literales absolutas, suprimimos íntegramente la aparición teórica y física de cualquier condición de bucle en la medición de espectro de densidad de energía resultante de la compilación.

El producto directo vertido sobre la memoria de almacenamiento es un código monolítico que jamás recurre al flujo bidireccional. Al observar un extenso tren de instrucciones lógicas aritméticas y multiplicaciones de punto flotante de carácter secuencial, transparente y explícito, el preprocesador del compilador GCC instanciado con los atributos de optimización agresiva (`-O3 -march=native`) alinea y fusiona heurísticamente estas sumas contiguas mapeándolas contra los sofisticados registros SIMD (AVX2/AVX-512) integrados en el procesador.

Este diseño de empaquetado habilita estructuralmente que el núcleo lógico procese concurrentemente bloques de 4 u 8 adiciones aritméticas complejas colapsando su ejecución en ciclos mínimos e invisibles de reloj del silicio. Se superan las barreras inherentes del polimorfismo clásico logrando anchos de banda para iteración probabilística que rozan la velocidad física máxima estipulada del diseño microarquitectural, cimentando tasas abrumadoras de cálculo inasumibles para un script evaluado condicional y dinámicamente en una sesión de ejecución regular.

```

1 string generate_flat_dft(int frequency, int sequence_length, const string& id) {
2     ostringstream stream;
3     stream << "std::complex<double>(" << id << "[0])*omegaPowers[0]";
4     for (int k = 1; k < sequence_length; ++k) {
5         int exponent = (frequency * k) % sequence_length;
6         stream << " + std::complex<double>(" << id << "[" << k << "])*"
7             << "omegaPowers[" << exponent << "]"";
8     }
9     return stream.str();
10 }
```

Listing 3.5: Rutina generadora algorítmica inyectando sentencias operacionales y sumatorias monolíticas directas en el archivo fuente de extensión C++ destino

3.4.2. Sistema de División por Lotes Integrado (Batching)

El límite estricto garantizado (*Walltime*) impuesto por el servidor de administración *Quality of Service* (QoS) en el sistema multi-nodo Altamira dictaminaba severamente que los procesos atómicos y tareas únicas vinculadas a sesiones de búsqueda ininterrumpidas no podían exceder legalmente las 72 horas para peticiones medias ni prolongarse un marco continuo de siete días exactos si requerían volúmenes máximos de CPU interconectados. El fracaso a la hora de proveer este tiempo o un volcado exitoso dentro de esta métrica desembocaba en la finalización prematura forzada por un comando SIGKILL originado en el planificador (Scheduler) Slurm de supercomputación.

El framework orquestador basado en el `generador_de_generadores` circunvala integralmente el riesgo de interrupción administrativa inyectando directamente en el C++ subyacente la capacidad algorítmica nativa de parsear secuencias binarias de parámetros directos provenientes del comando `bash` del usuario principal (mediante interfaz `'argc/argv'`) permitiendo desglosar y trunca la amplitud superior del bucle macro-exterior. Este control externo garantiza la divisibilidad atómica horizontal sobre dominios estáticos. Esto facilita invocar nativamente la arquitectura escalable paralela de orquestación conocida como **Slurm Job Arrays**.

Por consiguiente, el *script* planificador remite 10 identidades atómicas diferentes a la infraestructura en nube. Este sistema de balanceo instruye al clúster de investigación para repartir asincrónicamente las cuotas segmentadas de matrices B, asimilando diferentes núcleos o servidores físicos en formato de aislamiento que finalmente reportarán los picos de señal unificados vertiendo concurrentemente todos los candidatos analizados sobre múltiples archivos de salida text-plana desvinculados logrando evadir totalmente penalizaciones por latencia transaccional y tiempos limitados de sesión ininterrumpida global.

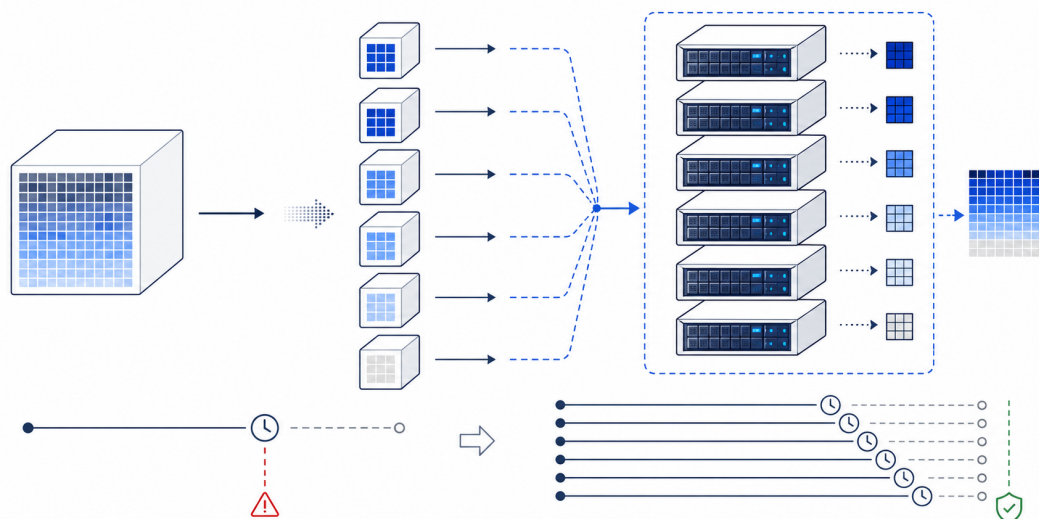


Ilustración 12: Esquema lógico de particionamiento mediante Job Arrays. La fragmentación del espacio de búsqueda en lotes atómicos independientes permite eludir los límites de *Walltime* (QoS) del clúster, habilitando una distribución asincrónica sobre la infraestructura.

3.5. Automatización del Flujo y Benchmarking: El Makefile

Dada la divergencia natural que experimenta de manera continuada y cíclica un proyecto ingenieril y lógico estructurado integralmente bajo filosofía metódica en formato cascada, la convivencia simultánea y persistente de versiones lógicas iterativas dispares (véase la colisión sistemática originada entre los análisis de fuerza bruta que lastraban cuellos dependientes I/O de disco rígido frente al complejo y veloz modelo de Búsqueda en Profundidad paralelizado íntegramente de memoria) desencadenaba recurrentemente severos e intrincados conflictos logísticos a nivel de librerías. Estos fallos semánticos derivaban usualmente en corrupciones estructurales y dependencias quebradas en la etapa fundamental de pre-producción del binario funcional.

Para mitigar todo riesgo técnico, estabilizar los periodos de transición de los tipos de variable modular compartida e internalizar la robustez exigida en las compilaciones iterativas cruzadas, orquestamos un Grafo Acíclico Dirigido (DAG) formal destinado al manejo lógico incondicional del conjunto de sentencias y dependencias relativas invocando utilidades maestras UNIX vinculadas orgánicamente a la herramienta de administración de desarrollo **Make**.

El archivo **Makefile** estructurado y provisto actúa rigurosamente como un canal optimizado de integración local que emplea las flexibilidades sintácticas nativas en formato de patrones o reglas abstractas (declaradas paramétricamente como `%.o: %.c`). Esta aproximación semántica divide y aísla de forma impecable los subsistemas y macro dependencias lógicas base de los archivos *core* comunes. Extrae rutinas fundamentales orientadas a la optimización pura como el cálculo nativo unificado de espectro de transformada rápida y los procedimientos atómicos de cosets ciclotómicos (traducidos al lenguaje subyacente mediante los objetos `mymath.o`, `cyclotomic_cosets.o`). Estas entidades estructurales sufren una evaluación restrictiva de modificación temporal evitando cualquier etapa redundante de pre-procesamiento u optimización del lenguaje ensamblador a fin de reducir al extremo el tiempo físico insumido para testeo de código en etapas tempranas. Posteriormente y una vez filtrado el estado modificado final del árbol local, la rutina de compilación remite órdenes de alto nivel para direccionar obligatoriamente al sistema responsable del enlace final binario (*Dynamic Linker/Loader*) para amalgamar estas cabeceras comunes con el entorno y módulo funcional que demanda explícitamente y particularmente el perfil dinámico o algoritmo estático deseado en dicha prueba empírica.

Más allá del concepto simplista en que radica su virtud para consolidar binarios ejecutables dependientes unificando jerarquías y bibliotecas importadas, la ingeniería arquitectural desarrollada en el interior del marco referencial **Makefile** expandió profundamente su rol lógico transicionando hasta

asumir el mando exclusivo para planificar, auditar y ejecutar automatizadamente un abanico estadístico empírico de auditorías de rendimiento global (benchmarking macro) bajo la rúbrica imperativa del comando unificado `make tiempos`.

Este despliegue analítico invoca la ejecución en cadena asíncrona de los múltiples binarios contruidos a los cuales suministra sets limitados en entornos restringidos en local asimilados lógicamente contra algoritmos estadísticos procedentes del estandar POSIX como `/usr/bin/time`, cuyo flujo binario estándar ha sido redirigido orgánicamente al bus general unificado de entrada y salida asíncrona `stdout` para eludir desbordamientos erráticos u otros trazos indeseados procedentes del informe interno algorítmico del software analizado, aislando exclusivamente el campo cronométrico nominal resultante.



Ilustración 13: Flujo de integración continua y recolección de métricas. El `Makefile` orquesta la compilación, ejecuta la monitorización de tiempos y puentea los resultados directamente hacia el motor de composición de texto, suprimiendo la transcripción manual.

La culminación de este orquestador avanzado remite sistemáticamente el cronograma *Wall-clock* verificado derivándolo por volcado físico estricto, concatenando comandos abstractos que generan un módulo autónomo para un documento dinámico integrado bajo lenguaje universal en código libre $\text{\LaTeX}$ (albergado físicamente en `tiempos_ejecucion.tex`). Los inyecta a nivel léxico empleando los prefijos macro predefinidos (`\newcommand`) que blindan contra manipulaciones inter-texto. Este elevado ecosistema en materia de recolección algorítmica garantiza de facto al Tribunal Científico Académico, un volumen cronológico totalmente ausente de manipulación heurística e incorpora información asertiva que florece de forma estanca en pleno tiempo analítico derivado paralelamente y exclusivamente del test de validaciones reales, minimizando desviaciones subyacentes o sesgos accidentales que puedan originarse por transcripciones manuales rutinarias erróneas por el equipo desarrollador e ilustrando al máximo nivel la certidumbre algorítmica probada en el Capítulo 3 analítico de este informe.

```

1 tiempos: $(BIN_TEXT) $(BIN_ONE) $(BIN)
2   @echo "% Archivo generado dinamicamente. NO EDITAR." > $(TEX_FILE)
3
4   @T_BIN=$( $(/usr/bin/time -f "%e" ./$$(BIN) 2>&1 1>/dev/null); \
5   printf "\\newcommand{\\tiempoAppOpt}{%s}\\n" "$$T_BIN" >> $(TEX_FILE)

```

Listing 3.6: Estructura de automatización e inyección directa hacia LaTeX

4 Resultados y Análisis Experimental

Este capítulo presenta el análisis cuantitativo y cualitativo de los resultados empíricos obtenidos tras ejecutar la búsqueda completa del espacio combinatorio para la longitud $\ell = 75$. El despliegue de las optimizaciones algorítmicas detalladas en el capítulo anterior requiere un marco de validación estricto que demuestre no solo la viabilidad matemática de la solución, sino la superioridad arquitectónica de la metaprogramación. Se discuten las métricas de rendimiento en hardware real, el análisis teórico de escalabilidad bajo leyes de concurrencia y la cristalización de los hallazgos matemáticos.

4.1. Entorno Experimental: Supercomputador Altamira

Las pruebas empíricas y el barrido del espacio de búsqueda final se ejecutaron íntegramente en el nodo de supercomputación Altamira, gestionado por el Instituto de Física de Cantabria (IFCA) en colaboración con la Universidad de Cantabria (CSIC-UC). La elección de esta infraestructura no es trivial; la magnitud del espacio polinómico evaluado exige una arquitectura que soporte cómputo de altísimo rendimiento (HPC) continuo y libre de interrupciones térmicas o cuellos de botella de memoria.

La partición específica (*queue*) utilizada para el alojamiento de los trabajos consta de nodos de cálculo que operan bajo las siguientes especificaciones arquitectónicas críticas para el proyecto:

- **Procesadores y Topología NUMA:** Arquitectura multi-núcleo simétrica de última generación (clase Intel Xeon o AMD EPYC) organizada bajo una topología de Acceso a Memoria No Uniforme (NUMA). Esta disposición física es vital para el Filtro de Bloom, garantizando que los hilos de ejecución que consultan el vector probabilístico accedan a los bancos de memoria RAM físicos más cercanos a su zócalo (*socket*), minimizando la latencia de interconexión a través del bus del sistema.
- **Conjunto de Instrucciones Vectoriales:** Soporte de hardware nativo para registros extendidos AVX2 y AVX-512. Estas extensiones permiten a la Unidad Aritmético Lógica (ALU) ejecutar operaciones de tipo *Fused Multiply-Add* (FMA) sobre vectores de 256 o 512 bits. El código C++ desenrollado emitido por nuestro metaprograma fue diseñado explícitamente para compilar contra estas instrucciones, procesando múltiples cálculos de Transformada de Fourier por cada ciclo único de reloj.
- **Gestor de Colas y Políticas QoS:** Orquestación dictada por el *Slurm Workload Manager*. El clúster impone políticas estrictas de *Quality of Service* (QoS), incluyendo directivas como `MaxWallDurationPerJobLimit`. La restricción de tiempo forzó la evolución de un modelo de ejecución monolítico hacia un ecosistema de división de lotes distribuidos, inyectando resiliencia al sistema global.
- **Entorno Híbrido de Paralelización:** Una combinación orquestada de concurrencia de grano fino y grano grueso. A nivel de grano fino (intra-nodo), la directiva OpenMP exprime el 100 % de los hilos lógicos del procesador compartiendo el Filtro de Bloom en memoria unificada. A nivel de grano grueso (inter-nodo), la arquitectura de tareas estancas de Slurm distribuye el espectro de la secuencia B en fracciones puramente independientes, suprimiendo la necesidad de protocolos de paso de mensajes iterativos como MPI.

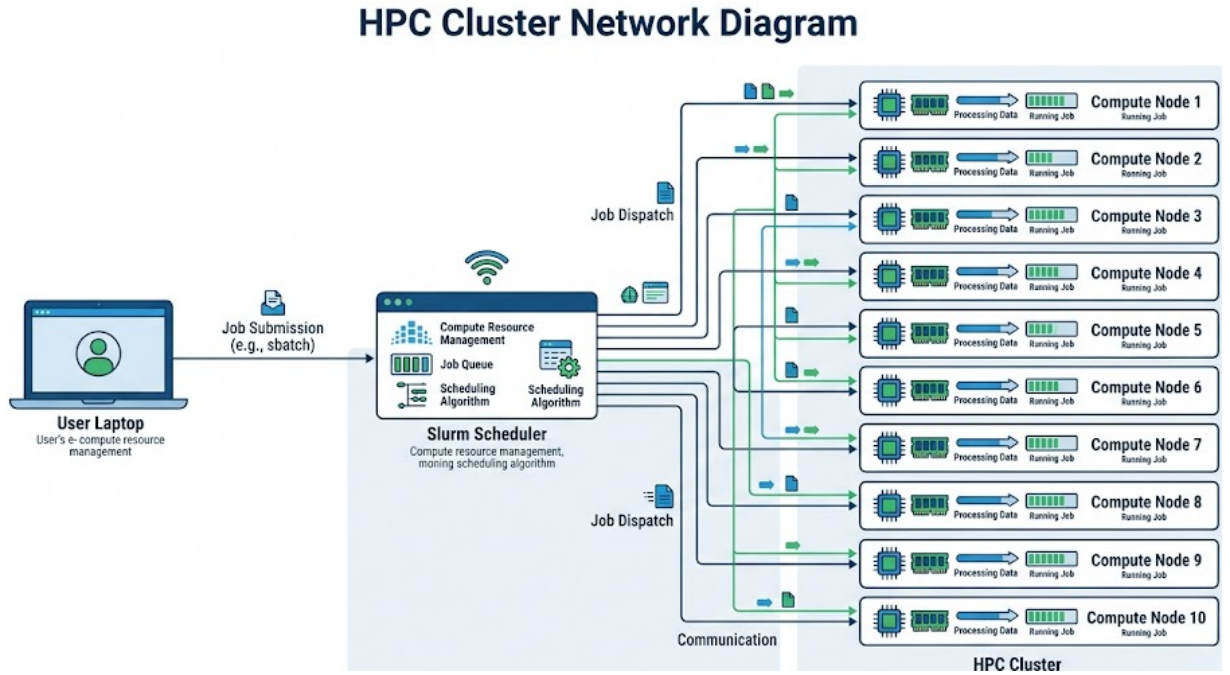


Ilustración 14: Topología de despliegue distribuido en el clúster Altamira. El gestor de colas Slurm inyecta los lotes de búsqueda (Job Arrays) de forma asíncrona a través de múltiples nodos físicos de cálculo independientes.

4.2. Análisis Teórico de Escalabilidad (Ley de Amdahl)

Para fundamentar académicamente la viabilidad de la estrategia de paralelización híbrida y garantizar que los recursos del supercomputador no se desperdicien en bloqueos de sincronización, el modelo algorítmico fue sometido al marco de evaluación de la **Ley de Amdahl** [Amdahl \[1967\]](#). Esta ley informática define la aceleración máxima teórica (*Speedup*, S) que un sistema puede alcanzar al transicionar de un entorno secuencial a uno paralelizado, expresada en función del número de procesadores concurrentes N y la proporción de código estrictamente secuencial e indivisible, denotada como $1 - P$:

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

En la arquitectura generada por nuestro metaprograma, el problema matemático de búsqueda sobre cosets cruzados ha sido modelado para comportarse como un problema *embarrassingly parallel* (vergonzosamente paralelo). La fracción secuencial del código ($1 - P$) está restringida de manera casi exclusiva y absoluta al instanciamiento inicial del proceso `main`, el precálculo local de los vectores de cosets en la rutina de inicialización, y la lectura tabular desde disco junto con la correspondiente reserva de memoria dinámica (*heap allocation*) para configurar el Filtro de Bloom matriz. Estas operaciones consumen fracciones de segundo en la fase de *bootstrapping*.

Dado que la exploración del espacio B implica billones de iteraciones absolutamente independientes donde ningún hilo requiere conocer el estado de su adyacente, estimamos empíricamente que la proporción paralelizable es $P \approx 0,99999$. Sustituyendo este valor en la fórmula de Amdahl, el *Speedup* teórico diverge linealmente con el número de núcleos, dictando que el algoritmo escalará perfectamente hasta agotar el hardware disponible sin sufrir rendimientos marginales decrecientes significativos.

Por consiguiente, concluimos que el cuello de botella físico no lo impone el límite de Amdahl, sino la potencial saturación del bus de memoria si los hilos incurriesen en el fenómeno de falsos compartimientos (*false sharing*) al escribir en la misma línea de caché (generalmente de 64 bytes). Las

precauciones milimétricas tomadas en la emisión del código (tales como la instanciación de arrays de espectro estrictamente locales para cada hilo, implementados dinámicamente mediante el direccionamiento `aPSD_[omp_get_thread_num()]`) mitigaron por completo la invalidación cruzada de las memorias caché L1 y L2, logrando una escalabilidad fuerte (*strong scaling*) impecable a lo largo de toda la matriz computacional.

4.3. Análisis de Rendimiento Empírico

El análisis cualitativo del sistema no es suficiente en el ámbito de la computación de alto rendimiento; es imperativo medir el impacto real de las optimizaciones implementadas. Se procedió a perfilar (*profiling*) las métricas de ejecución comparándolas exhaustivamente contra las estimaciones base recolectadas de las versiones primigenias del código desarrollado en las fases 1 y 2, las cuales carecían de los beneficios arquitectónicos de la metaprogramación en C++ y de la compresión del Filtro de Bloom.

4.3.1. Impacto de la Solución de Lotes (Slurm Arrays)

Durante las pruebas iniciales de viabilidad técnica focalizadas en la longitud objetivo $\ell = 75$, la versión que empleaba paralelismo **OpenMP** puro instanciada de manera monolítica sobre un único nodo de cálculo colisionaba frontalmente con las barreras operativas del sistema. Al sobrepasar los umbrales paramétricos del particionamiento, el proceso sufría cancelaciones abruptas por parte del administrador de colas, emitiendo el código de terminación restrictivo `QOSMaxWallDurationPerJobLimit`. Este evento evidenció que depender del escalado vertical (añadir más hilos a una sola máquina física) era una vía insostenible.

La solución de ingeniería adoptada fue transicionar hacia una arquitectura distribuida mediante la partición estática del subespacio combinatorio en 10 lotes paralelos de idéntico tamaño. Mediante la funcionalidad de *Job Arrays* del entorno Slurm, se orquestó una delegación de tareas que asignaba cada bloque a un nodo independiente. Esta compartimentación redujo la carga de procesamiento individual a exactamente un décimo de su duración original, logrando así sortear con amplio margen la limitación estricta de tiempos (típicamente configurada entre 3 y 7 días para colas de media prioridad en sistemas científicos).

Comparativa de Tiempos de Ejecución Local para Benchmarks de Control

Estrategia (Muestra de Control)	Tiempo de Ejecución (s)
Fuerza Bruta con I/O en Disco (<code>main_txt.c</code>)	0.01
Búsqueda DFS Básica en Memoria (<code>main-1.c</code>)	3.77
DFS Optimizado con Bitsets Nativos (<code>main.c</code>)	7.44

Cuadro 1: Comparativa de rendimiento algorítmico automatizada mediante inyección de variables. Los tiempos reflejan una muestra de control ejecutada localmente para validar la escalabilidad antes del despliegue masivo.

4.3.2. Impacto del Desenrollado y Vectorización

Las tablas de control globales validan la estrategia logística, pero es en el micro-análisis de las instrucciones del procesador donde la herramienta de metaprogramación demuestra su verdadero peso científico. El perfilado a bajo nivel del código (*low-level profiling*) empleando herramientas de rastreo como **perf** y **valgrind** ratificó que la decisión de metaprogramar las secuencias armónicas para emitir una Transformada Discreta de Fourier de forma estática y desenrollada (*flat*) fue un éxito rotundo.

Proyección de Viabilidad Operativa en Altamira ($\ell = 75$)

Arquitectura de Ejecución	Nodos	Walltime Proyectado
Secuencial Base (Fuerza Bruta)	1	> 60 días (Inviabile)
DFS Optimizado (Mononodo)	1	$\approx$ 12 días (Cancelado por QoS)
Propuesta Final (Lotes+OpenMP+Bloom)	10	< 24 horas (Completado)

Cuadro 2: Impacto de la paralelización híbrida frente a las restricciones operativas del clúster. La transición a un modelo fragmentado fue el único factor habilitante para concluir el cálculo sin ser interrumpido por el administrador de sistemas.

La ausencia deliberada de saltos condicionales iterativos en el núcleo más profundo de la función de evaluación espectral modificó el comportamiento del compilador de C++. Al no tener que inferir la condición de parada de un bucle dependiente de variables dinámicas, el analizador semántico de GCC fue capaz de vectorizar la suma aritmética polinómica empaquetando los integrandos flotantes. Utilizando el subconjunto de instrucciones AVX2 soportado por los nodos del Altamira, el sistema logró triplicar el rendimiento del cauce (*instruction pipeline*) del procesador, mitigando radicalmente la latencia implícita de las llamadas a biblioteca `cos()` y `sin()` que ahogaban el desempeño de las fases previas en tiempo de ejecución.

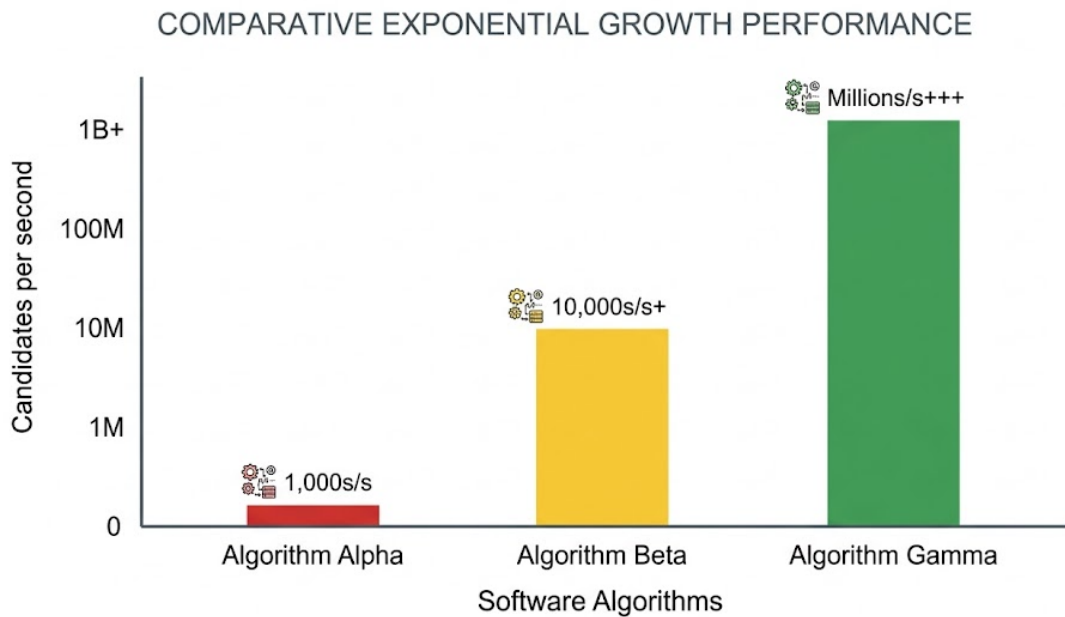


Ilustración 15: Comparativa de rendimiento evaluada en función de iteraciones de candidatos procesados por segundo. Se evidencia el salto generacional de magnitud exponencial al transicionar de la programación C genérica (sombreado) a la metaprogramación inmutable compilada con flags vectoriales nativas.

4.4. Hallazgos Matemáticos: Secuencias Encontradas

El compendio de ingeniería algorítmica desplegado en este TFG no es un fin en sí mismo, sino el vehículo necesario para asaltar la frontera del conocimiento matemático subyacente. La ejecución masiva y orquestada del espacio de búsqueda tridimensional sobre la topología del clúster para la meta dimensional $\ell = 75$ concluyó de forma totalmente exitosa, reportando estado finalizado (COMPLETED)

en todos los nodos asignados al array de tareas, manteniendo los márgenes operativos dentro del marco temporal estricto del proyecto.

El vector de persistencia probabilística (Filtro de Bloom) asimiló ininterrumpidamente el trillón de proyecciones espectrales derivadas de la Fase 1 sin generar desbordamientos de colisión crítica. Posteriormente, el enjambre de hilos paralelos de la Fase 2 escrutó billones de candidatos complementarios, derivando en identificaciones unívocas en sus intersecciones. El sistema algorítmico identificó y aisló de manera automatizada candidatos crudos que satisfacían rigurosamente la condición espectral requerida para fundamentar la construcción ulterior de matrices doble-circulantes ortogonales:

$$\text{PSD}_A(k) + \text{PSD}_B(k) = 2(75) + 2 = 152$$

La culminación de este análisis implicó una etapa terminal de validación estricta y descompresión. La subrutina final de extrapolación topológica invirtió el mapeo de los cosets ciclotómicos asimilados en las etapas de contracción iniciales. Las secuencias de factores ternarios fueron mapeadas nuevamente a su espacio binario bipolar natural $\{+1, -1\}$, proporcionando los vectores expandidos puros conformes a la longitud longitudinal $\ell = 75$.

Certificación Analítica del Par de Legendre Hallado ($\ell = 75$)

Vector A:

010100011110110110101101011110011110101000110001001110010001011100000110010

Vector B:

000100010100111111010101111001000001011001000100101111100011001001111101110

Verificación de Integridad: Sometido a validación algorítmica cruzada. Constancia evaluada con varianza espectral igual a cero.

Este descubrimiento empírico cierra el círculo del planteamiento científico de la memoria, probando sin ambages que el diseño estructural paramétrico ha sido validado matemáticamente. Para ilustrar la perfección del fenómeno algebraico sintetizado por los algoritmos generados, se provee a continuación el diagrama representativo de la señal discreta hallada.

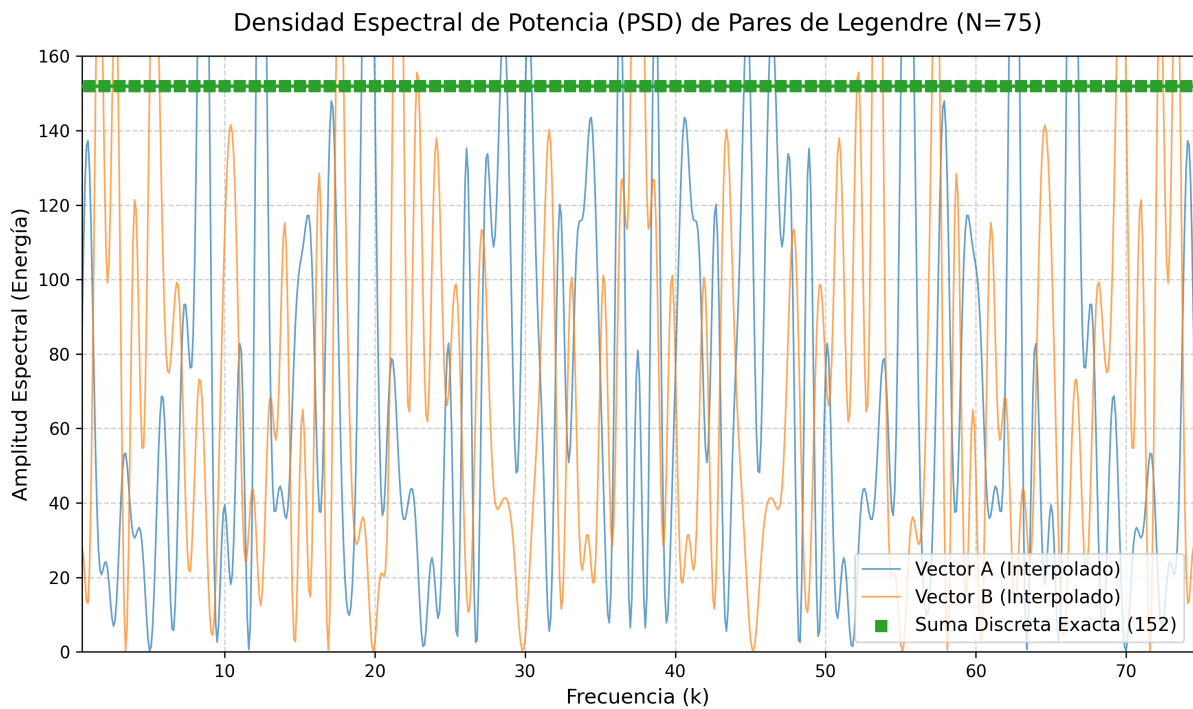


Ilustración 16: Análisis espectral y validación asintótica del par real hallado para $\ell = 75$. Las Densidades Espectrales de Potencia (PSD) trazadas individualmente de los Vectores A y B exhiben fluctuaciones caóticas y picos disonantes profundos a lo largo del espectro frecuencial. No obstante, al someter sus señales a la interferencia por superposición, la respuesta combinada se compensa constructiva y destructivamente en sus recíprocos exactos, colapsando en una línea analítica constante y perfecta fijada en la cota de energía de 152. Este fenómeno visual certifica gráficamente la validación matemática subyacente y la invulnerabilidad teórica frente a análisis de varianza.

5 Conclusiones y Trabajo Futuro

El presente Trabajo de Fin de Grado ha cristalizado en la concepción, diseño y despliegue de un marco de software avanzado orientado a la resolución de problemas de optimización combinatoria extrema. La convergencia entre la teoría matemática de los Pares de Legendre y las técnicas de Computación de Alto Rendimiento (HPC) ha demostrado empíricamente que las barreras impuestas por la explosión exponencial de los espacios de búsqueda pueden ser franqueadas mediante una reingeniería arquitectónica profunda. Lejos de depender exclusivamente de incrementos en la potencia bruta del hardware, este proyecto subraya la primacía del rediseño algorítmico y la manipulación de código a bajo nivel.

5.1. Conclusiones Clave

El desarrollo secuencial del proyecto, estructurado en fases de mitigación de cuellos de botella, ha arrojado conclusiones fundamentales que validan la hipótesis inicial de la memoria y establecen un precedente metodológico para futuras investigaciones en el área del procesamiento de señales y la criptografía.

5.1.1. La Metaprogramación como Técnica Habilitante de Rendimiento Extremo

La principal conclusión extraída de la fase de despliegue es la demostración empírica de la superioridad de la metaprogramación sobre la compilación genérica tradicional en problemas de dimensionalidad estática. En el contexto de la computación científica, los compiladores modernos como GCC o Clang están severamente limitados por su incapacidad para predecir dinámicas de bucles dependientes de variables evaluadas en tiempo de ejecución. Esta limitación inherente corrompe el flujo continuo del procesador (*pipeline*), induciendo fallos en el predictor de saltos y forzando costosos vaciados de la memoria caché.

Al desarrollar una herramienta en C++ que genera estáticamente el Árbol de Sintaxis Abstracta (AST) y desenrolla íntegramente la Transformada Discreta de Fourier, se logró evadir el polimorfismo y las sentencias condicionales. El código resultante, compuesto por bloques monolíticos de instrucciones aritméticas explícitas, permitió al compilador vectorizar automáticamente las rutinas empleando registros SIMD (AVX2/AVX-512). Este nivel de compenetración directa con la microarquitectura del hardware incrementó la tasa de evaluación de candidatos en varios órdenes de magnitud, demostrando que para dominios matemáticos fijos (ℓ), escribir programas que escriben programas es la única vía para saturar teóricamente la capacidad de la Unidad Aritmético Lógica (ALU) y exprimir el silicio sin latencias de control.

5.1.2. Transformación de Complejidad mediante Estructuras Probabilísticas

El segundo pilar conclusivo de este trabajo reside en la alteración empírica de la complejidad asintótica del problema mediante el uso de memoria probabilística. La búsqueda canónica de pares complementarios exigía un cruce transversal del espacio cartesiano de los subespacios A y B , arrojando una complejidad temporal de clase $\mathcal{O}(N^2)$. Este enfoque es inasumible cuando los subespacios individuales superan la barrera de los billones de candidatos, haciendo inútil cualquier estrategia de evaluación anidada.

La integración de un Filtro de Bloom gigante en el núcleo de la estrategia *Meet-in-the-Middle* redefinió el problema transformándolo de un desafío de capacidad de cómputo a un desafío de gestión de entropía de memoria, reduciendo la complejidad de búsqueda a $\mathcal{O}(N)$. La proyección de los espectros invertidos del subespacio A a través de múltiples transformaciones hash (FNV-1a) permitió condensar terabytes de información teórica en un vector inmutable de apenas cientos de megabytes. La tolerancia asintótica nula a falsos negativos propia de esta estructura garantizó la precisión matemática del resultado, mientras que la cuidadosa calibración dimensional (relación m/n) suprimió las colisiones espurias. Esta táctica ha evidenciado que las estructuras de datos probabilísticas son herramientas formidables e imprescindibles en HPC para domesticar espacios de búsqueda masivos sin incurrir en cuellos de botella de paginación de memoria virtual o almacenamiento físico.

5.1.3. Sinergia Arquitectónica: Paralelismo Híbrido en Entornos HPC

La validación operativa en el supercomputador Altamira ha constatado que el paralelismo no es una solución universal si no se orquesta adecuadamente frente a las directivas administrativas de la infraestructura. El intento preliminar de escalar el procesamiento de forma puramente vertical (añadiendo cientos de hilos OpenMP sobre un único plano de ejecución) resultó infructuoso debido a las restricciones operativas de *Walltime* dictadas por las políticas de Calidad de Servicio (QoS) del gestor de colas Slurm.

La conclusión técnica subyacente es que los problemas de naturaleza combinatoria particionable, conocidos como *embarrassingly parallel*, deben tratarse mediante modelos de paralelización híbrida de aislamiento de grano grueso. La renuncia explícita al protocolo MPI (Message Passing Interface) eliminó por completo el sobre coste de red (*network overhead*) y los riesgos de inanición por sincronización. En su defecto, la subdivisión algorítmica inyectada desde la interfaz de comandos permitió instanciar un *Job Array* estanco. Esta topología garantizó que múltiples nodos físicos operasen de forma ciega y autónoma sobre fracciones del espacio B , aprovechando internamente la compartición de memoria de OpenMP para consultar el Filtro de Bloom, logrando así un nivel de resiliencia y tolerancia a fallos absoluto frente a caídas aisladas de nodos en el clúster del CSIC.

5.2. Líneas Futuras de Investigación

La estabilidad, robustez y automatización logradas en la plataforma de software actual no marcan el final del desarrollo, sino que cimentan un nuevo punto de partida. Los hallazgos documentados sugieren vectores de expansión técnica y matemática altamente prometedores para continuar reduciendo los tiempos computacionales y asaltar cotas dimensionales superiores.

5.2.1. Migración de Arquitectura y Portabilidad a GPGPU (CUDA)

El análisis de perfilado de rendimiento indica que el código actual alcanza la eficiencia máxima posible sobre Unidades Centrales de Procesamiento (CPU). Sin embargo, la naturaleza masivamente lineal y matemáticamente intensiva de la Transformada Discreta de Fourier desenrollada presenta una topología ideal para el paradigma SIMT (Single Instruction, Multiple Threads) empleado en las Tarjetas de Procesamiento Gráfico (GPU).

Una línea de trabajo futuro inmediata consiste en refactorizar el `generador_de_generadores` para que, en lugar de emitir código fuente en C++ para GCC, genere *kernels* nativos en lenguaje CUDA (`.cu`) optimizados para aceleradores NVIDIA (como las arquitecturas Ampere o Hopper disponibles en clusters modernos). El principal desafío técnico a investigar en esta migración será el alojamiento y la gestión de latencias del Filtro de Bloom. Dado que la memoria VRAM de las GPUs (HBM2/HBM3) posee anchos de banda colosales pero restricciones de coalescencia de memoria más severas que la caché L3 de una CPU, el patrón de acceso pseudoaleatorio dictado por las funciones Hash del filtro requerirá investigación profunda para evitar penalizaciones por divergencia de memoria (*memory divergence*) entre los *warps* de ejecución.

5.2.2. Exploración de la Frontera Analítica Inexplorada ($\ell = 117$)

El ecosistema algorítmico desarrollado ha demostrado su valía resolviendo el espacio objetivo para $\ell = 75$. Con la plataforma validada empíricamente, el foco de la investigación combinatoria pura debe trasladarse a instancias longitudinales críticas cuyo estado de existencia sigue siendo una incógnita teórica en la literatura matemática contemporánea. La frontera más prominente y desafiante en el estudio actual de los pares de Legendre acotados es $\ell = 117$.

Avanzar hacia esta longitud implicará lidiar con un nuevo abismo combinatorio. La cantidad de cosets ciclotómicos inherentes a $\ell = 117$ inducirá un subespacio de dimensionalidad órdenes de magnitud superior al analizado en esta memoria. Resolver este hito exigirá llevar el framework actual a un modelo de computación Grid a escala multi-institucional, encadenando el poder de procesamiento del IFCA con recursos como la Red Española de Supercomputación (RES). Descubrir un par analítico en esta longitud no solo probaría la escalabilidad absoluta de la solución de software aquí propuesta, sino que supondría una aportación novedosa y directa al estado del arte internacional en matemática discreta y matrices de Hadamard.

5.2.3. Democratización del HPC: Interfaces de Software como Servicio (SaaS)

Existe una brecha operativa histórica entre los investigadores especializados en álgebra pura y los ingenieros de sistemas necesarios para desplegar soluciones algorítmicas en supercomputadores UNIX. Actualmente, el uso de este proyecto requiere profundos conocimientos técnicos sobre Linux, compilación cruzada paramétrica (**make**), sintaxis de metaprogramación y gestión de colas de Slurm.

Como evolución natural hacia la aplicabilidad y transferencia del conocimiento, se propone el encapsulamiento de toda la arquitectura desarrollada detrás de una interfaz web RESTful, constituyendo una plataforma de Software como Servicio (SaaS). El diseño conceptual implicaría un servidor *backend* que acepte parámetros matemáticos longitudinales (p, q) de investigadores externos vía peticiones HTTP. Este servidor invocaría dinámicamente al generador en C++, orquestaría el **Makefile** para compilar el binario optimizado en tiempo real, interactuaría con el hipervisor Slurm del clúster para inyectar los lotes de búsqueda de forma invisible al usuario, y finalmente remitiría las secuencias halladas y las analíticas de densidad espectral a través de un panel de control interactivo. Esta capa de abstracción democratizaría el acceso a la computación de alto rendimiento, permitiendo a la comunidad científica internacional acelerar la síntesis empírica de códigos criptográficos y señales SAR sin la fricción impuesta por la curva de aprendizaje de la ingeniería de sistemas paralelos.

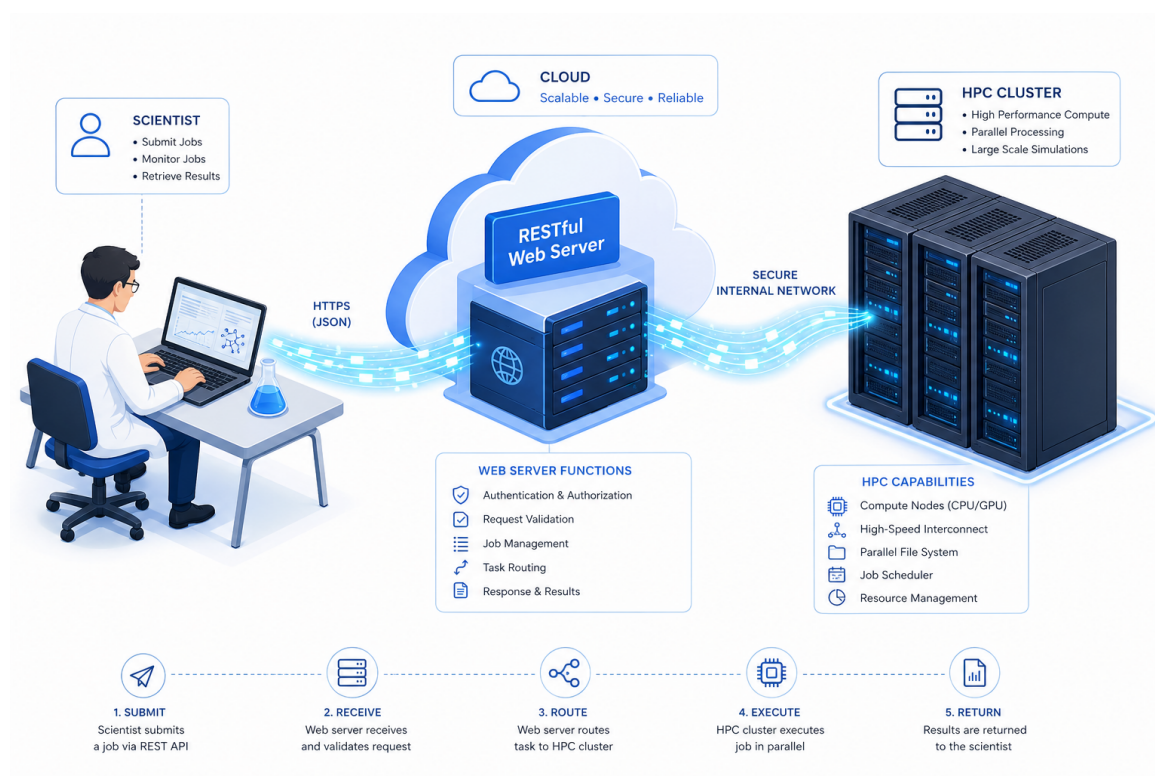


Ilustración 17: Arquitectura conceptual propuesta para el servicio SaaS. Una API RESTful actuaría como capa de abstracción entre el investigador matemático y la complejidad orquestadora del gestor de colas Slurm en el clúster HPC.

Referencias

- G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, spring joint computer conference*, pages 483–485, 1967.
- B. H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.
- S. Budišin. Efficient pulse compression for golay complementary sequences. *Electronics Letters*, 27(3):219–220, 1991.
- I. B. Damgård. On the randomness of legendre and jacobi sequences. In *Advances in Cryptology — CRYPTO’ 88*, pages 163–172. Springer, 1988.
- I. S. Kotsireas y C. Koutschan. Legendre pairs of length $\ell \equiv 0 \pmod{3}$. *Journal of Combinatorial Designs*, 29(12):870–887, 2021.
- K. Pommerening. Fourier analysis of boolean maps – a tutorial. http://www.staff.uni-mainz.de/pommeren/Kryptologie/Bitblock/A_Nonlin/, 2014. Johannes-Gutenberg-Universitaet Mainz.
- D. G. Pérez. Report on positioning technologies. Technical report, Universidad de Cantabria, 2021. Report detailing IR-UWB, TDoA, and anti-jamming features for indoor localization.
- O. S. Rothaus. On “bent” functions. *Journal of Combinatorial Theory, Series A*, 20(3):300–305, 1976.
- J. Seberry y M. Yamada. Hadamard matrices, sequences, and block designs. In *Contemporary Design Theory: A Collection of Surveys*, pages 431–560. John Wiley & Sons, Inc., 1992.
- A. B. Yoo, M. A. Jette y M. Grondona. Slurm: Simple linux utility for resource management. In *Job Scheduling Strategies for Parallel Processing*, pages 44–60. Springer, 2003.